

# CONSUMO ABUSIVO DE SPA EN JÓVENES DE BOGOTÁ: UN ANÁLISIS SOCIODEMOGRÁFICO DESDE LOS DATOS DE VESPA (2019-2026)

Sara Gabriela Cajamarca Lozano

Santiago Herrera León

Raúl Felipe Ropero Barbosa

Dumar Alexis Ruiz Bernal

Valeria Vélez Molina

Universidad Ean

Bogotá, Colombia

2026

## 1. Planteamiento del problema

### 1.1. Descripción del problema

El consumo abusivo de sustancias psicoactivas (SPA) en adolescentes y jóvenes es un problema de salud pública que compromete el desarrollo social, educativo y emocional de esta población. En Bogotá, la Secretaría Distrital de Salud registra estos casos a través del subsistema VESPA (Vigilancia Epidemiológica del Consumo de Sustancias Psicoactivas), lo que permite construir un panorama detallado sobre dónde, cómo y en qué condiciones se presenta este fenómeno.

Sin embargo, este panorama suele quedar disperso en reportes administrativos que no siempre se traducen en un análisis sistemático que identifique patrones claros y accionables. Este proyecto busca cerrar esa brecha, analizando los datos abiertos de VESPA bajo la siguiente pregunta problema:

**¿Qué características sociodemográficas y de contexto (lugar habitual de consumo, nivel educativo, aseguramiento en salud) predominan en los casos de consumo abusivo de sustancias psicoactivas en adolescentes y jóvenes de Bogotá, reportados al sistema VESPA entre 2019 y 2026?**

Para responderla de forma ordenada, se plantearon cinco preguntas de apoyo:

1. ¿Cómo se distribuyen los casos de consumo abusivo de SPA en adolescentes y jóvenes según la localidad de residencia en Bogotá?
2. ¿Cómo ha evolucionado el número de casos reportados en jóvenes a lo largo de los años del periodo (2019-2026)?
3. ¿Qué diferencias hay entre hombres y mujeres jóvenes en el número de casos reportados?
4. ¿En qué lugares habituales de consumo se concentran más los casos de jóvenes?
5. ¿Qué relación existe entre el nivel educativo de los jóvenes y el tipo de aseguramiento en salud de los casos reportados?

Adicionalmente, se planteó una pregunta orientada a la construcción de un modelo predictivo:

**A partir del perfil sociodemográfico de un caso (sexo, nivel educativo, tipo de aseguramiento, lugar habitual de consumo y año), ¿es posible predecir si ese caso proviene de una localidad de alto riesgo (Kennedy, Bosa, Suba, Ciudad Bolívar o Engativá, las cinco localidades con más casos)?**

## 2. Objetivo General

Analizar las características sociodemográficas y de contexto que predominan en los casos de consumo abusivo de SPA en adolescentes y jóvenes de Bogotá, a partir de los datos del sistema VESPA (2019-2026), con el fin de identificar patrones de riesgo que orienten estrategias de prevención.

### 2.1. Objetivos Específicos

- Depurar y preparar la base de datos de VESPA para aislar el subconjunto correspondiente a adolescentes y jóvenes en el periodo de estudio.
- Explorar la distribución de los casos según localidad, año, sexo, lugar habitual de consumo y nivel educativo.
- Describir estadísticamente la variable de conteo CASOS y evaluar, mediante pruebas de hipótesis, la asociación entre variables sociodemográficas clave y la concentración territorial del consumo abusivo.
- Construir y comparar modelos de clasificación que permitan predecir si un caso proviene de una localidad de alto riesgo, a partir de su perfil sociodemográfico.
- Formular conclusiones y recomendaciones de política pública a partir de los hallazgos, en línea con el Objetivo de Desarrollo Sostenible (ODS) 12: Producción y consumo responsables.

### 3. Marco de Referencia

El proyecto se enmarca en el Objetivo de Desarrollo Sostenible 12, Producción y consumo responsables (Naciones Unidas, s.f.), y busca aportar evidencia que apoye el diseño de políticas públicas de prevención dirigidas a jóvenes.

## 4. Búsqueda y Obtención de los Datos

### 4.1. Fuente de los Datos

Los datos utilizados provienen del subsistema VESPA (Vigilancia Epidemiológica del Consumo de Sustancias Psicoactivas), administrado por la Secretaría Distrital de Salud de Bogotá, y publicados de manera abierta a través del portal Datos Abiertos Bogotá / Observatorio SaluData (Secretaría Distrital de Salud de Bogotá, s.f.).

### 4.2. Justificación de la Fuente

Se eligió esta fuente por las siguientes razones:

- **Es un registro oficial:** proviene directamente de la autoridad sanitaria distrital, lo que garantiza respaldo institucional y trazabilidad de la información.
- **Es de acceso público:** cumple con los principios de datos abiertos, lo que permite su uso académico e investigativo sin restricciones.
- **Se actualiza periódicamente:** la versión utilizada fue actualizada el 29 de julio de 2026 y contiene registros desde 2015 hasta junio de 2026, lo que asegura vigencia en los hallazgos.
- **Es pertinente al problema de investigación:** al ser un sistema de vigilancia epidemiológica específico para consumo de SPA, sus variables están directamente alineadas con la pregunta de investigación del proyecto y con el marco del ODS 12.

### 4.3. Alcance y Limitaciones de la Fuente

Es importante señalar que los datos de VESPA corresponden a casos reportados, es decir, personas que ingresaron a algún proceso de atención o notificación relacionado con consumo abusivo de SPA. El análisis describe entonces el universo de casos detectados por el sistema de salud, no necesariamente la totalidad del fenómeno en la población general, dado que existe un subregistro inherente a cualquier sistema de vigilancia epidemiológica.

## 5. Comprensión del Conjunto de Datos

### 5.1. Descripción General

El conjunto de datos original contiene 101.500 filas y 21 columnas, con registros desde 2015 hasta junio de 2026.

Para iniciar el análisis, se carga el archivo CSV en un DataFrame de pandas y se hace una primera inspección visual de las columnas y sus valores.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import scipy.stats as st
from IPython.display import display

sns.set_style('whitegrid')
pd.set_option('display.max_columns', 30)

df = pd.read_csv('osb_saludmental_-consumoabusivo_spageneral.csv', sep=';', encoding='utf-8-sig')
df.head(10)
```

Out[1]:

|   | ANO  | SEXO   | NOMBRELOCALIDADRESIDENCIA | MESNOTIFICACION | TRIMESTRE | TIPOASEGURAMIENTO | SITIOHABITUALCONSUMO_VIVIENDA | SITIOHABITUA |
|---|------|--------|---------------------------|-----------------|-----------|-------------------|-------------------------------|--------------|
| 0 | 2022 | Hombre | Kennedy                   | 6               | 2         | Subsidiado        | NO                            |              |
| 1 | 2022 | Hombre | Mártires                  | 6               | 2         | Subsidiado        | SI                            |              |
| 2 | 2022 | Hombre | Kennedy                   | 6               | 2         | Contributivo      | NO                            |              |
| 3 | 2022 | Hombre | Kennedy                   | 6               | 2         | Contributivo      | NO                            |              |
| 4 | 2022 | Mujer  | Kennedy                   | 6               | 2         | Contributivo      | NO                            |              |
| 5 | 2022 | Hombre | Mártires                  | 6               | 2         | Subsidiado        | NO                            |              |
| 6 | 2022 | Hombre | Kennedy                   | 6               | 2         | Subsidiado        | NO                            |              |
| 7 | 2022 | Hombre | Mártires                  | 6               | 2         | Subsidiado        | NO                            |              |
| 8 | 2022 | Hombre | Sin dato                  | 6               | 2         | Subsidiado        | NO                            |              |
| 9 | 2022 | Hombre | Mártires                  | 6               | 2         | Subsidiado        | NO                            |              |

Esta primera vista confirma que el dataset trae columnas categóricas (sexo, localidad, tipo de aseguramiento) y columnas SI/NO para los lugares habituales de consumo, todas en formato texto.

5.2. Unidad de Análisis

Un aspecto fundamental para la correcta interpretación de este dataset es que una fila no representa una persona. Cada fila corresponde a una combinación única de categorías (año, sexo, localidad, tipo de aseguramiento, entre otras), y la columna `CASOS` indica cuántos casos reales corresponden a esa combinación específica. Esta particularidad tiene implicaciones directas sobre cómo se aborda tanto el análisis estadístico como la construcción del modelo predictivo.

5.3. Descripción de las Variables

La Tabla 1 resume las variables del conjunto de datos VESPA, con base en el diccionario de variables publicado por la Secretaría Distrital de Salud de Bogotá.

In [2]:

```
tabla1_variables = pd.DataFrame([
    ("ANO", "Año de notificación del caso"),
    ("SEXO", "Sexo reportado (Hombre, Mujer, Intersexual, Sin dato)"),
    ("NOMBRELOCALIDADRESIDENCIA", "Localidad de residencia en Bogotá (21 localidades)"),
    ("MESNOTIFICACION / TRIMESTRE", "Mes y trimestre de notificación"),
    ("TIPOASEGURAMIENTO", "Tipo de afiliación al sistema de salud (Contributivo, Subsidiado, Vinculado, Excepción, Particular, Régimen especial)"),
    ("SITIOHABITUALCONSUMO_* (6 columnas)", "Indicadores SI/NO del lugar habitual de consumo: vivienda, parque, colegio, bares/tabernas, vía públ:
    ("NIVELEDCATIVO", "Último nivel educativo alcanzado"),
    ("CURSO_DE_VIDA", "Grupo etario (Adolescencia, Juventud, Adultez, Vejez)"),
    ("COMOACUDIOTRATAMIENTO", "Forma en la que el caso llegó a tratamiento"),
    ("PERTENENCIAIETNICA", "Pertenencia étnica reportada"),
    ("ORIENTSEXUAL", "Orientación sexual reportada"),
    ("PAISNACIONALIDAD", "País de nacionalidad"),
    ("NOMBREUPZ", "Unidad de Planeamiento Zonal (subdivisión geográfica dentro de la localidad)"),
    ("ESTADOCIVIL", "Estado civil reportado"),
    ("CASOS", "Número de casos reales que agrupa esa combinación de categorías"),
], columns=["Variable", "Descripción"])

display(tabla1_variables)
```

|    | Variable                            | Descripción                                       |
|----|-------------------------------------|---------------------------------------------------|
| 0  | ANO                                 | Año de notificación del caso                      |
| 1  | SEXO                                | Sexo reportado (Hombre, Mujer, Intersexual, Si... |
| 2  | NOMBRELOCALIDADRESIDENCIA           | Localidad de residencia en Bogotá (21 localida... |
| 3  | MESNOTIFICACION / TRIMESTRE         | Mes y trimestre de notificación                   |
| 4  | TIPOASEGURAMIENTO                   | Tipo de afiliación al sistema de salud (Contri... |
| 5  | SITIOHABITUALCONSUMO_* (6 columnas) | Indicadores SI/NO del lugar habitual de consum... |
| 6  | NIVELEDUCATIVO                      | Último nivel educativo alcanzado                  |
| 7  | CURSO_DE_VIDA                       | Grupo etario (Adolescencia, Juventud, Adultez,... |
| 8  | COMOACUDIOTRATAMIENTO               | Forma en la que el caso llegó a tratamiento       |
| 9  | PERTENENCIAETNICA                   | Pertenencia étnica reportada                      |
| 10 | ORIENTSEXUAL                        | Orientación sexual reportada                      |
| 11 | PAISNACIONALIDAD                    | País de nacionalidad                              |
| 12 | NOMBREUPZ                           | Unidad de Planeamiento Zonal (subdivisión geog... |
| 13 | ESTADOCIVIL                         | Estado civil reportado                            |
| 14 | CASOS                               | Número de casos reales que agrupa esa combinac... |

### 5.4. Calidad Inicial de los Datos

Para conocer el tamaño real del dataset y su estado de calidad inicial, se revisan las filas, columnas, nulos por columna, duplicados exactos y la distribución de `CURSO_DE_VIDA`.

```
In [3]: print("Filas:", df.shape[0], " Columnas:", df.shape[1])
print("\nNulos por columna:\n", df.isnull().sum())
print("\nDuplicados exactos:", df.duplicated().sum())
print("\nCategorías en CURSO_DE_VIDA:\n", df['CURSO_DE_VIDA'].value_counts())
```

Filas: 101500 Columnas: 21

Nulos por columna:

|                                     |     |
|-------------------------------------|-----|
| ANO                                 | 0   |
| SEXO                                | 44  |
| NOMBRELOCALIDADRESIDENCIA           | 0   |
| MESNOTIFICACION                     | 0   |
| TRIMESTRE                           | 0   |
| TIPOASEGURAMIENTO                   | 0   |
| SITIOHABITUALCONSUMO_VIVIENDA       | 0   |
| SITIOHABITUALCONSUMO_PARQUE         | 0   |
| SITIOHABITUALCONSUMO_EST_EDUCATIVO  | 0   |
| SITIOHABITUALCONSUMO_BARES_TABERNAS | 0   |
| SITIOHABITUALCONSUMO_VIA_PUBLICA    | 0   |
| SITIOHABITUALCONSUMO_CASA_AMIGOS    | 0   |
| NIVELEDUCATIVO                      | 0   |
| CURSO_DE_VIDA                       | 0   |
| COMOACUDIOTRATAMIENTO               | 0   |
| PERTENENCIAETNICA                   | 106 |
| ORIENTSEXUAL                        | 0   |
| PAISNACIONALIDAD                    | 0   |
| NOMBREUPZ                           | 0   |
| ESTADOCIVIL                         | 0   |
| CASOS                               | 0   |

dtype: int64

Duplicados exactos: 0

Categorías en CURSO\_DE\_VIDA:

|               |       |
|---------------|-------|
| CURSO_DE_VIDA |       |
| Juventud      | 41058 |
| Adultez       | 38817 |
| Adolescencia  | 18084 |
| Vejez         | 3541  |

Name: count, dtype: int64

El resultado muestra que el dataset original tiene 101.500 filas y 21 columnas, con 150 valores nulos en total (44 en SEXO y 106 en PERTENENCIAETNICA), 0 duplicados exactos, y una distribución de CURSO\_DE\_VIDA de 41.058 filas en Juventud, 38.817 en Adultez, 18.084 en Adolescencia y 3.541 en Vejez. Los dos últimos grupos (Adolescencia y Juventud) son el foco de este proyecto.

### 6. Limpieza y Preparación de los Datos

Antes de iniciar el análisis, fue necesario resolver tres problemas en los datos crudos: valores de sin dato escritos de formas distintas, valores nulos en dos columnas, y la necesidad de aislar únicamente el subconjunto de interés (adolescentes y jóvenes en el periodo 2019-2026).

## 6.1. Unificación de Valores Nulos y "sin dato"

Se revisa si existen variantes mal escritas de sin dato (como Sin Dato, SIN DATO o N.A.) y se unifican todas bajo una sola etiqueta. Los valores nulos reales de SEXO y PERTENENCIAETNICA se completan también con la etiqueta Sin dato, en lugar de eliminarlos, para no perder información de las demás columnas de esas filas.

En este dataset específico no se encontraron variantes mal escritas de sin dato (no se necesitó reemplazo real en ese aspecto), pero sí se completaron correctamente los 150 valores nulos originales (44 en SEXO y 106 en PERTENENCIAETNICA).

Para aplicar esta unificación se ejecuto lo siguiente:

```
In [4]: valores_nulos_disfrazados = ['Sin dato', 'Sin Dato', 'SIN DATO', 'N.A.']
df_limpio = df.replace(valores_nulos_disfrazados, 'Sin dato')
df_limpio[['SEXO', 'PERTENENCIAETNICA']] = df_limpio[['SEXO', 'PERTENENCIAETNICA']].fillna('Sin dato')
print("Verificación NOMBRELOCALIDADRESIDENCIA:", df_limpio['NOMBRELOCALIDADRESIDENCIA'].unique())
```

Verificación NOMBRELOCALIDADRESIDENCIA: ['Kennedy' 'Mártires' 'Sin dato' 'Fontibón' 'Bosa' 'Puente Aranda' 'Engativá' 'Santa Fe' 'Usme' 'Ciudad Bolívar' 'Teusaquillo' 'Tunjuelito' 'San Cristóbal' 'Usaquén' 'Suba' 'Antonio Nariño' 'Chapinero' 'Rafael Uribe' 'Candelaria' 'Barrios Unidos' 'Sumapaz']

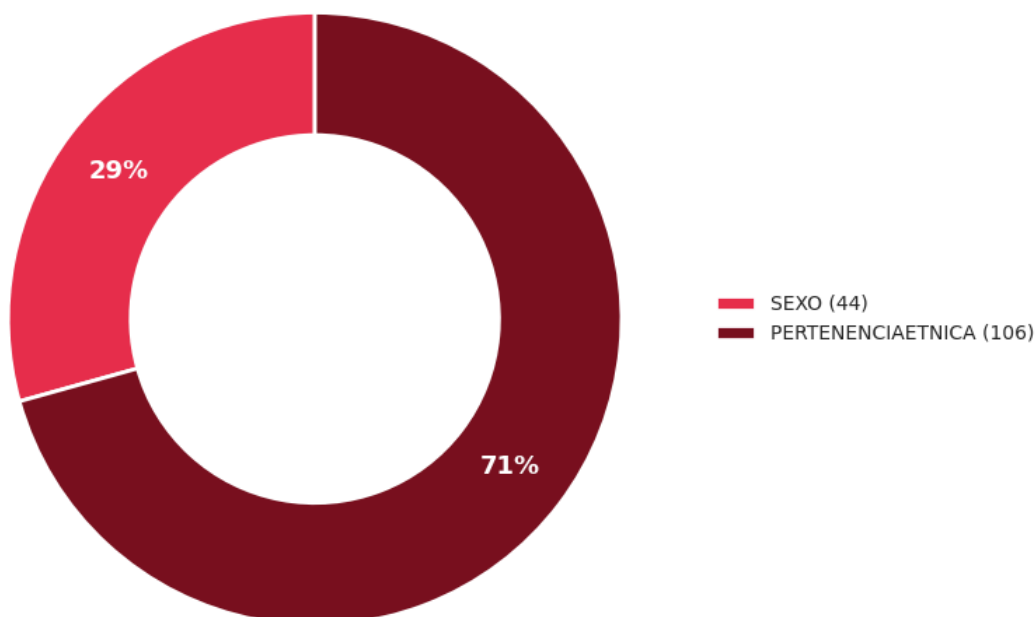
**Figura 1.** Distribución de los 150 valores nulos en el dato crudo.

```
In [5]: valores = [44, 106]
etiquetas = ['SEXO (44)', 'PERTENENCIAETNICA (106)']
colores = ['#E62E4C', '#7a1220']

fig, ax = plt.subplots(figsize=(8,6))
wedges, texts, autotexts = ax.pie(
    valores,
    colors=colores,
    autopct='%1.0f%%',
    startangle=90,
    pctdistance=0.8,
    wedgeprops={'width': 0.4, 'edgecolor': 'white', 'linewidth': 2},
    textprops={'fontsize': 13, 'fontweight': 'bold', 'color': 'white'}
)

ax.legend(wedges, etiquetas, loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10, frameon=False)
plt.title('Distribución de los 150 valores nulos en el dataset original', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

**Distribución de los 150 valores nulos en el dataset original**



El 71% de los valores nulos corresponde a PERTENENCIAETNICA (106 casos) y el 29% restante a SEXO (44 casos).

## 6.2. Filtrado del Alcance del Análisis

El dataset crudo incluye los cuatro grupos etarios de CURSO\_DE\_VIDA (Adolescencia, Juventud, Adultez, Vejez) y registros desde 2015. Como la pregunta de investigación se enfoca únicamente en adolescentes y jóvenes entre 2019 y 2026, se aplicó el siguiente filtro.

```
In [6]: df_jovenes = df_limpio[
        (df_limpio['CURSO_DE_VIDA'].isin(['Adolescencia', 'Juventud'])) &
        (df_limpio['ANO'] >= 2019) & (df_limpio['ANO'] <= 2026)
        ].copy()

print("Filas totales limpias:", len(df_limpio))
print("Filas de jóvenes 2019-2026:", len(df_jovenes))
print("Total de casos (suma de CASOS):", df_jovenes['CASOS'].sum())
```

Filas totales limpias: 101500

Filas de jóvenes 2019-2026: 38585

Total de casos (suma de CASOS): 41345

El resultado es que, de las 101.500 filas originales, quedan 38.585 filas correspondientes a adolescentes y jóvenes en el periodo de estudio, las cuales agrupan un total de 41.345 casos (recordando que CASOS puede ser mayor a 1 por fila). Esto significa que se removieron 62.915 filas en total: 42.358 de Adultez/Vejez, y 20.557 de jóvenes que estaban fuera del rango de años 2019-2026.

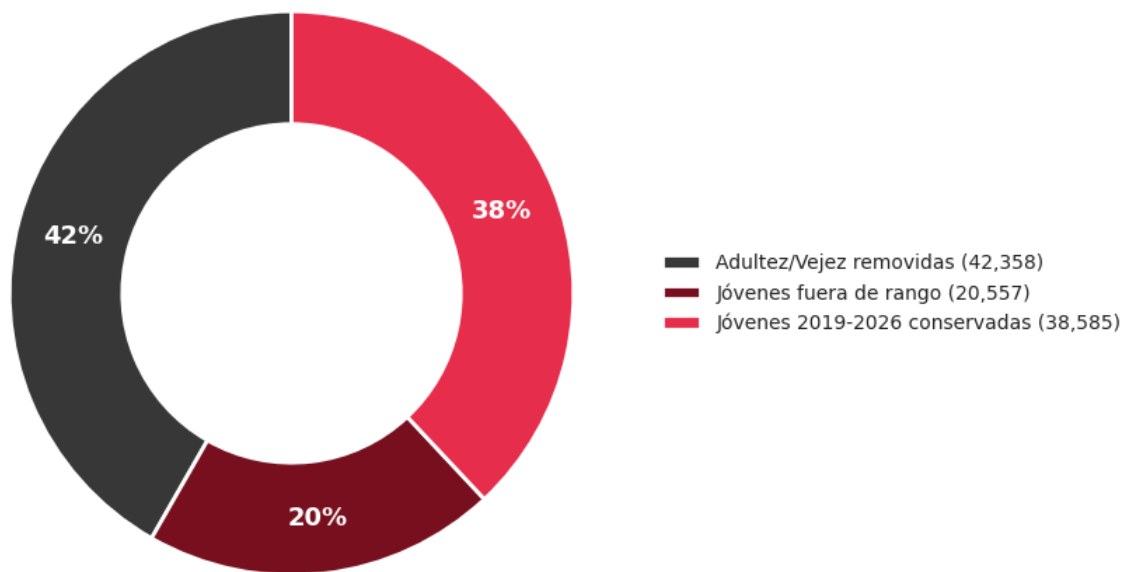
**Figura 2.** Filtrado del dataset de 101.500 filas totales.

```
In [7]: valores = [42358, 20557, 38585]
etiquetas = ['Adultez/Vejez removidas (42,358)', 'Jóvenes fuera de rango (20,557)', 'Jóvenes 2019-2026 conservadas (38,585)']
colores = ['#3a3a3a', '#7a1220', '#E62E4C']

fig, ax = plt.subplots(figsize=(8,6))
wedges, texts, autotexts = ax.pie(
    valores,
    colors=colores,
    autopct='%1.0f%%',
    startangle=90,
    pctdistance=0.8,
    wedgeprops={'width': 0.4, 'edgecolor': 'white', 'linewidth': 2},
    textprops={'fontsize': 13, 'fontweight': 'bold', 'color': 'white'})

ax.legend(wedges, etiquetas, loc='center left', bbox_to_anchor=(1, 0.5), fontsize=10, frameon=False)
plt.title('Filtrado del dataset: 101,500 filas totales', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

**Filtrado del dataset: 101,500 filas totales**



## 6.3. Verificación de la Limpieza

Para confirmar el efecto de la limpieza, se compara el estado de los datos antes y después del proceso.

```
In [8]: comparacion = pd.DataFrame({
    "Antes (Datos Crudos)": [df.shape[0], df.isnull().sum().sum(), df.duplicated().sum()],
    "Después (Jóvenes - Datos Limpios)": [len(df_jovenes), df_jovenes.isnull().sum().sum(), df_jovenes.duplicated().sum()]
}, index=["Filas", "Valores nulos", "Duplicados"])
display(comparacion)
```

|               | Antes (Datos Crudos) | Después (Jóvenes - Datos Limpios) |
|---------------|----------------------|-----------------------------------|
| Filas         | 101500               | 38585                             |
| Valores nulos | 150                  | 0                                 |
| Duplicados    | 0                    | 7                                 |

Los 7 duplicados que aparecen después de filtrar son filas que, tras quedarse solo con las columnas relevantes para adolescentes y jóvenes, coinciden exactamente entre sí. No se eliminan porque representan combinaciones de categorías legítimas, no errores de carga.

Finalmente, se verifica una muestra aleatoria de los datos ya limpios y se guarda el resultado en un nuevo archivo para su uso en las siguientes etapas del proyecto.

```
In [9]: df_jovenes.sample(10, random_state=1)
```

|       | ANO  | SEXO   | NOMBRELOCALIDADRESIDENCIA | MESNOTIFICACION | TRIMESTRE | TIPOASEGURAMIENTO | SITIOHABITUALCONSUMO_VIVIENDA | SITIOHAB |
|-------|------|--------|---------------------------|-----------------|-----------|-------------------|-------------------------------|----------|
| 25823 | 2024 | Mujer  | Bosa                      | 7               | 3         | Vinculado         | NO                            |          |
| 45817 | 2025 | Hombre | Chapinero                 | 10              | 4         | Contributivo      | SI                            |          |
| 20858 | 2024 | Hombre | Teusaquillo               | 3               | 1         | Excepción         | NO                            |          |
| 22509 | 2024 | Hombre | Teusaquillo               | 4               | 2         | Excepción         | NO                            |          |
| 52303 | 2026 | Hombre | Ciudad Bolívar            | 5               | 2         | Contributivo      | NO                            |          |
| 5245  | 2022 | Hombre | Kennedy                   | 12              | 4         | Contributivo      | NO                            |          |
| 91826 | 2021 | Mujer  | Suba                      | 7               | 3         | Contributivo      | SI                            |          |
| 81609 | 2020 | Mujer  | Tunjuelito                | 2               | 1         | Subsidiado        | NO                            |          |
| 16997 | 2023 | Hombre | Kennedy                   | 9               | 3         | Subsidiado        | NO                            |          |
| 23806 | 2024 | Hombre | Engativá                  | 5               | 2         | Contributivo      | NO                            |          |

```
In [10]: df_jovenes.to_csv('spa_jovenes_limpio.csv', index=False)
print("Archivo limpio guardado. Filas:", len(df_jovenes), " | Total casos:", df_jovenes['CASOS'].sum())
```

Archivo limpio guardado. Filas: 38585 | Total casos: 41345

Con esto, la base `df_jovenes` (38.585 filas, 41.345 casos) queda lista para el análisis exploratorio.

## 7. Análisis Exploratorio de Datos (EDA)

A continuación, se responden las cinco preguntas de apoyo planteadas al inicio del proyecto, usando la base `df_jovenes` ya limpia.

### 7.1. Pregunta 1: Distribución de casos por localidad

Esta primera pregunta busca ubicar geográficamente el problema: Identificar si el consumo abusivo de SPA en jóvenes se distribuye de forma pareja entre las 20 localidades de Bogotá, o si existe una concentración territorial específica.

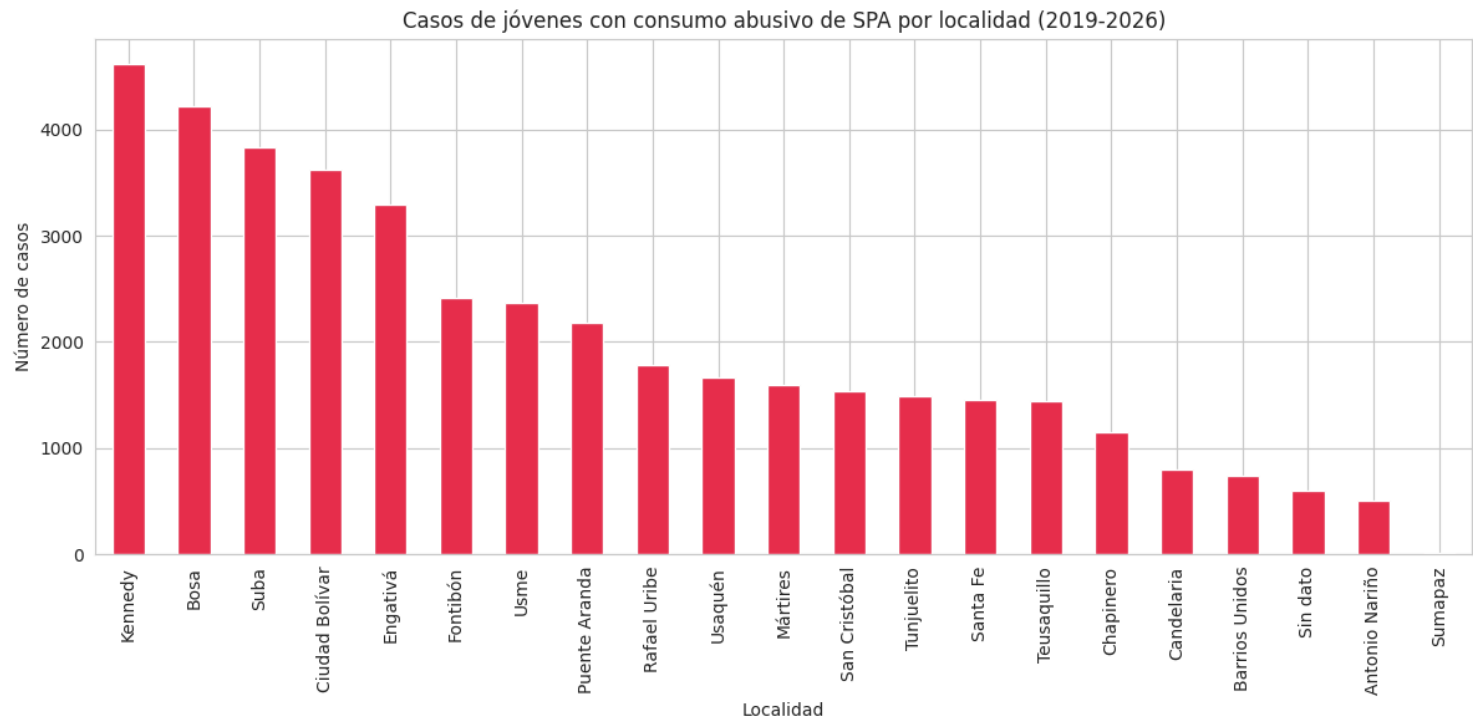
Para responder esta pregunta se agruparon los casos por localidad:

```
In [11]: total_casos = df_jovenes['CASOS'].sum()

casos_localidad = df_jovenes.groupby('NOMBRELOCALIDADRESIDENCIA')['CASOS'].sum().sort_values(ascending=False)

plt.figure(figsize=(12,6))
casos_localidad.plot(kind='bar', color='#E62E4C')
```

```
plt.title('Casos de jóvenes con consumo abusivo de SPA por localidad (2019-2026)')
plt.xlabel('Localidad')
plt.ylabel('Número de casos')
plt.tight_layout()
plt.show()
```



El resultado muestra una concentración marcada en pocas localidades. Para verlo con más detalle en proporción, se calculó también el porcentaje que representa cada una sobre el total de casos:

**Tabla 3.** Casos por localidad de residencia, en número y porcentaje.

```
In [12]: tabla3_localidad = pd.DataFrame({
    'Casos': casos_localidad,
    'Porcentaje (%)': (casos_localidad/total_casos*100).round(1)
})
display(tabla3_localidad)
```

|                           | Casos | Porcentaje (%) |
|---------------------------|-------|----------------|
| NOMBRELOCALIDADRESIDENCIA |       |                |
| Kennedy                   | 4624  | 11.2           |
| Bosa                      | 4223  | 10.2           |
| Suba                      | 3829  | 9.3            |
| Ciudad Bolívar            | 3628  | 8.8            |
| Engativá                  | 3294  | 8.0            |
| Fontibón                  | 2413  | 5.8            |
| Usme                      | 2369  | 5.7            |
| Puente Aranda             | 2182  | 5.3            |
| Rafael Uribe              | 1787  | 4.3            |
| Usaquén                   | 1662  | 4.0            |
| Mártires                  | 1596  | 3.9            |
| San Cristóbal             | 1541  | 3.7            |
| Tunjuelito                | 1492  | 3.6            |
| Santa Fe                  | 1452  | 3.5            |
| Teusaquillo               | 1445  | 3.5            |
| Chapinero                 | 1155  | 2.8            |
| Candelaria                | 802   | 1.9            |
| Barrios Unidos            | 738   | 1.8            |
| Sin dato                  | 599   | 1.4            |
| Antonio Nariño            | 505   | 1.2            |
| Sumapaz                   | 9     | 0.0            |

Las cinco primeras localidades (Kennedy, Bosa, Suba, Ciudad Bolívar y Engativá) concentran el 47.4% de todos los casos, a pesar de representar apenas 5 de las 20 localidades de Bogotá. Esta concentración no es aleatoria: todas ellas pertenecen al sur y suroccidente de Bogotá, zonas que históricamente presentan mayores índices de necesidades básicas insatisfechas y menor cobertura de servicios sociales.

En el extremo opuesto, localidades como Antonio Nariño (1.2%) o Sumapaz (prácticamente nula, por ser rural y de baja densidad poblacional) muestran que el fenómeno no se distribuye de forma pareja en el territorio, sino que responde a un patrón geográfico y socioeconómico identificable. Este hallazgo es la base para definir, más adelante, la variable objetivo del modelo predictivo: se agrupan estas cinco localidades bajo la categoría de alto riesgo precisamente porque aquí queda demostrado, con datos, que concentran casi la mitad del problema.

## 7.2. Pregunta 2: Evolución de los casos por año

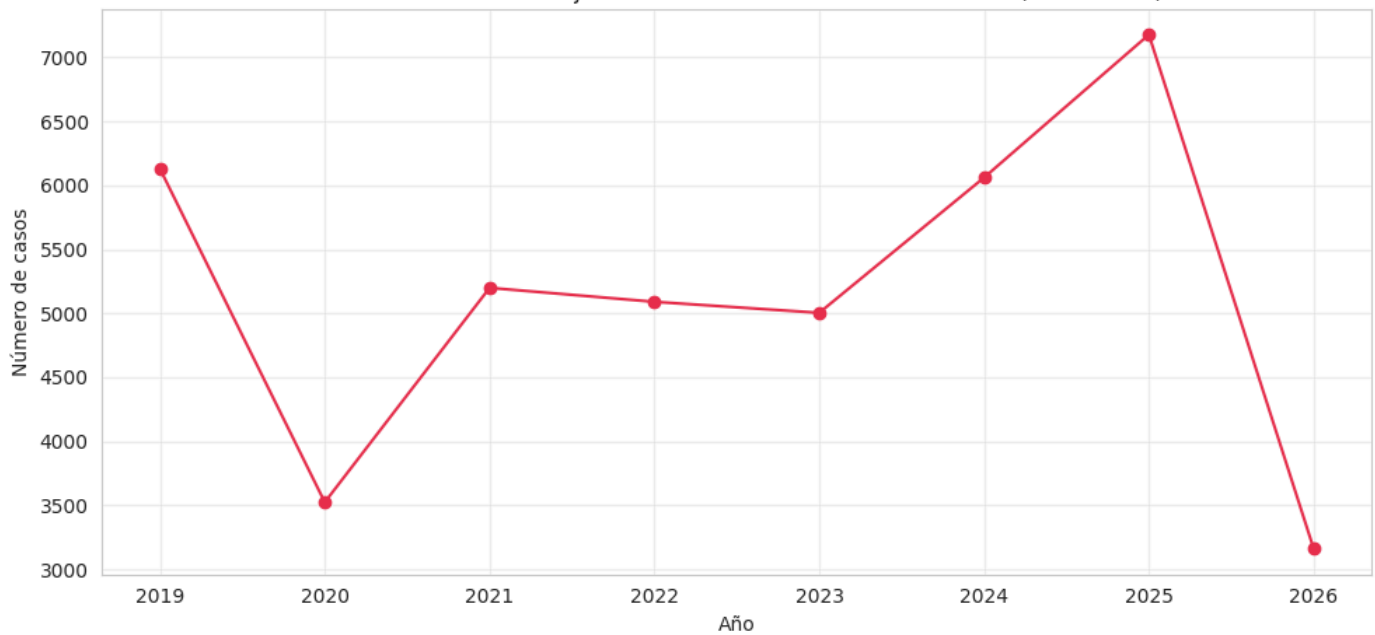
Aquí se analiza la dimensión temporal del fenómeno, para saber si los casos han aumentado, disminuido o se han mantenido estables a lo largo de los ocho años que cubre el periodo de estudio.

Para ver la evolución temporal se agruparon los casos por año:

```
In [13]: casos_anio = df_jovenes.groupby('ANO')['CASOS'].sum().sort_index()

plt.figure(figsize=(10,5))
casos_anio.plot(kind='line', marker='o', color='#E62E4C')
plt.title('Evolución de casos de jóvenes con consumo abusivo de SPA (2019-2026)')
plt.xlabel('Año')
plt.ylabel('Número de casos')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

Evolución de casos de jóvenes con consumo abusivo de SPA (2019-2026)



La serie temporal revela tres momentos distintos. Primero, una caída marcada en 2020 (3.514 casos, el valor más bajo del periodo), que coincide con las restricciones de movilidad y el menor acceso a servicios de salud presenciales durante la pandemia, lo que probablemente redujo la notificación de casos más que el consumo real.

Segundo, una recuperación sostenida entre 2021 y 2024, que no solo alcanza sino que supera los niveles prepandemia, sugiriendo que el problema no desapareció, solo dejó de registrarse temporalmente.

Tercero, un pico marcado en 2025 con 7.176 casos, el valor más alto de todo el periodo analizado, casi el doble que en 2020. La caída aparente en 2026 no debe leerse como una mejora: los datos de ese año solo cubren hasta junio, por lo que en términos anualizados la tendencia de crecimiento observada desde 2021 probablemente continúa. Esta lectura importa porque, de las variables analizadas en el modelo predictivo, el año de notificación termina siendo la más influyente, lo que confirma que el fenómeno tiene una dinámica temporal real y no estática.

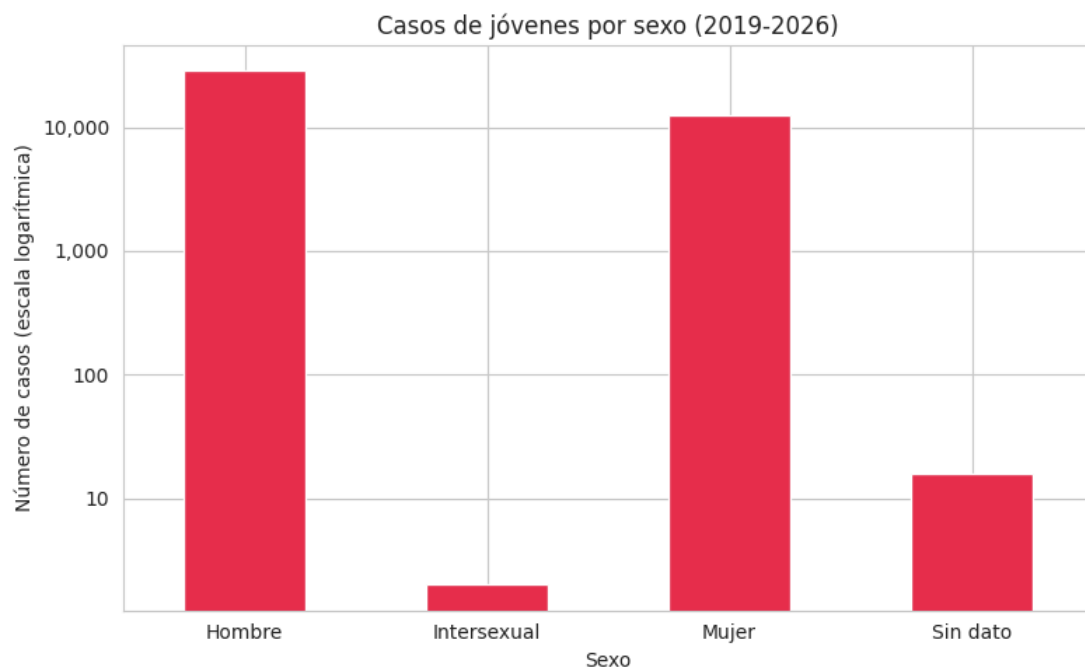
### 7.3. Pregunta 3: Diferencias por sexo

Esta pregunta explora si el consumo abusivo afecta de manera similar a hombres y mujeres jóvenes, o si existe una brecha significativa entre ambos grupos.

Para comparar hombres y mujeres se agruparon los casos por sexo:

```
In [14]: casos_sexo = df_jovenes.groupby('SEXO')['CASOS'].sum()

plt.figure(figsize=(8,5))
casos_sexo.plot(kind='bar', color=['#E62E4C'])
plt.title('Casos de jóvenes por sexo (2019-2026)')
plt.xlabel('Sexo')
plt.ylabel('Número de casos (escala logarítmica)')
plt.yscale('log')
plt.minorticks_off()
plt.xticks(rotation=0)
ax = plt.gca()
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x):,}'))
plt.tight_layout()
plt.show()
```



**Tabla 4.** Casos por sexo, en número y porcentaje.

Se usó escala logarítmica en el eje vertical del gráfico porque la diferencia entre categorías es demasiado grande para visualizarse en escala normal.

```
In [15]: tabla4_sexo = pd.DataFrame({
    'Casos': casos_sexo,
    'Porcentaje (%)': (casos_sexo/total_casos*100).round(1)
})
display(tabla4_sexo)
```

|                    | Casos | Porcentaje (%) |
|--------------------|-------|----------------|
| <b>SEXO</b>        |       |                |
| <b>Hombre</b>      | 28679 | 69.4           |
| <b>Intersexual</b> | 2     | 0.0            |
| <b>Mujer</b>       | 12648 | 30.6           |
| <b>Sin dato</b>    | 16    | 0.0            |

La diferencia por sexo es la más marcada de las cinco preguntas de apoyo: los hombres reportan 28.679 casos frente a 12.648 en mujeres, es decir, 2.3 veces más. Esta proporción (69.4% frente a 30.6%) es consistente con patrones reportados en otros estudios de consumo de sustancias en población joven en Latinoamérica, como la Encuesta Nacional de Consumo de Sustancias Psicoactivas (DANE, s.f.) y los reportes del Observatorio de Drogas de Colombia (ODC, s.f.), donde factores como la exposición social al consumo, las normas de género en torno al riesgo y el acceso diferenciado a espacios de socialización nocturna suelen explicar brechas similares.

Para la política pública, esto no significa que el consumo en mujeres deba ignorarse (12.648 casos siguen siendo una cifra considerable), sino que las estrategias de prevención más generales, dirigidas a la población masculina joven, probablemente tengan mayor retorno en términos de casos evitados, sin dejar de mantener rutas de atención específicas para mujeres, que pueden enfrentar barreras distintas para buscar ayuda.

## 7.4. Pregunta 4: Lugares habituales de consumo

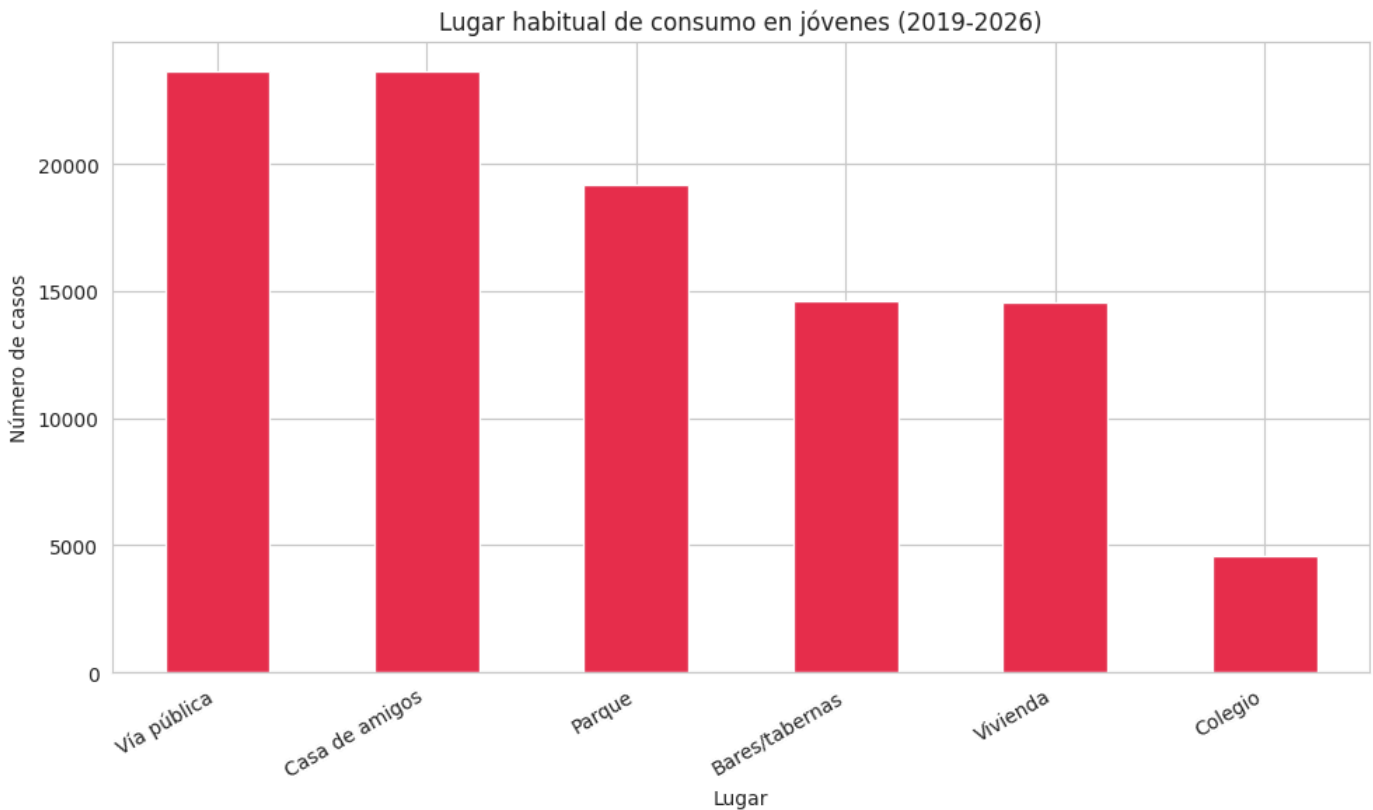
Se busca identificar en qué entornos ocurre con mayor frecuencia el consumo, información clave para orientar en qué espacios físicos deberían enfocarse las estrategias de prevención. Como cada joven puede reportar más de un lugar habitual de consumo, se suman los casos marcados como SI en cada una de las seis columnas de sitio.

```
In [16]: etiquetas_sitio = {
    'SITIOHABITUALCONSUMO_VIVIENDA': 'Vivienda',
    'SITIOHABITUALCONSUMO_PARQUE': 'Parque',
    'SITIOHABITUALCONSUMO_EST_EDUCATIVO': 'Colegio',
    'SITIOHABITUALCONSUMO_BARES_TABERNAS': 'Bares/tabernas',
    'SITIOHABITUALCONSUMO_VIA_PUBLICA': 'Vía pública',
    'SITIOHABITUALCONSUMO_CASA_AMIGOS': 'Casa de amigos'
}

totales_sitio = {}
for col, etiqueta in etiquetas_sitio.items():
    totales_sitio[etiqueta] = df_jovenes.loc[df_jovenes[col] == 'SI', 'CASOS'].sum()
```

```
serie_sitio = pd.Series(totales_sitio).sort_values(ascending=False)

plt.figure(figsize=(10,6))
serie_sitio.plot(kind='bar', color='#E62E4C')
plt.title('Lugar habitual de consumo en jóvenes (2019-2026)')
plt.xlabel('Lugar')
plt.ylabel('Número de casos')
plt.xticks(rotation=30, ha='right')
plt.tight_layout()
plt.show()
```



**Tabla 5.** Casos por lugar habitual de consumo, en número y porcentaje. Cada joven puede reportar más de un lugar habitual de consumo, por lo que los porcentajes no suman 100%.

```
In [17]: tabla5_sitio = pd.DataFrame({
    'Casos': serie_sitio,
    'Porcentaje (%)': (serie_sitio/total_casos*100).round(1)
})
display(tabla5_sitio)
```

|                       | Casos | Porcentaje (%) |
|-----------------------|-------|----------------|
| <b>Vía pública</b>    | 23635 | 57.2           |
| <b>Casa de amigos</b> | 23635 | 57.2           |
| <b>Parque</b>         | 19167 | 46.4           |
| <b>Bares/tabernas</b> | 14633 | 35.4           |
| <b>Vivienda</b>       | 14575 | 35.3           |
| <b>Colegio</b>        | 4590  | 11.1           |

La vía pública y la casa de amigos empatan como los entornos más comunes de consumo (57.2% cada uno), seguidos de cerca por los parques (46.4%). El colegio, en contraste, aparece en último lugar con apenas 11.1%. Este orden es contraintuitivo frente a la percepción común de que el consumo juvenil de SPA ocurre principalmente en el entorno escolar, y tiene una implicación directa de política pública: si las estrategias de prevención se concentran en el colegio, como suele ocurrir, estarían apuntando al entorno con menor incidencia real, dejando sin cobertura a los tres espacios donde realmente se concentra el fenómeno: la calle, las casas de amigos y los parques, todos ellos espacios de socialización no supervisada por una institución.

Esto sugiere que las políticas de intervención deberían trasladarse del aula hacia estrategias de ocupación del tiempo libre, programas comunitarios de barrio, y presencia institucional en espacios públicos, más que reforzar únicamente los programas de prevención escolar.

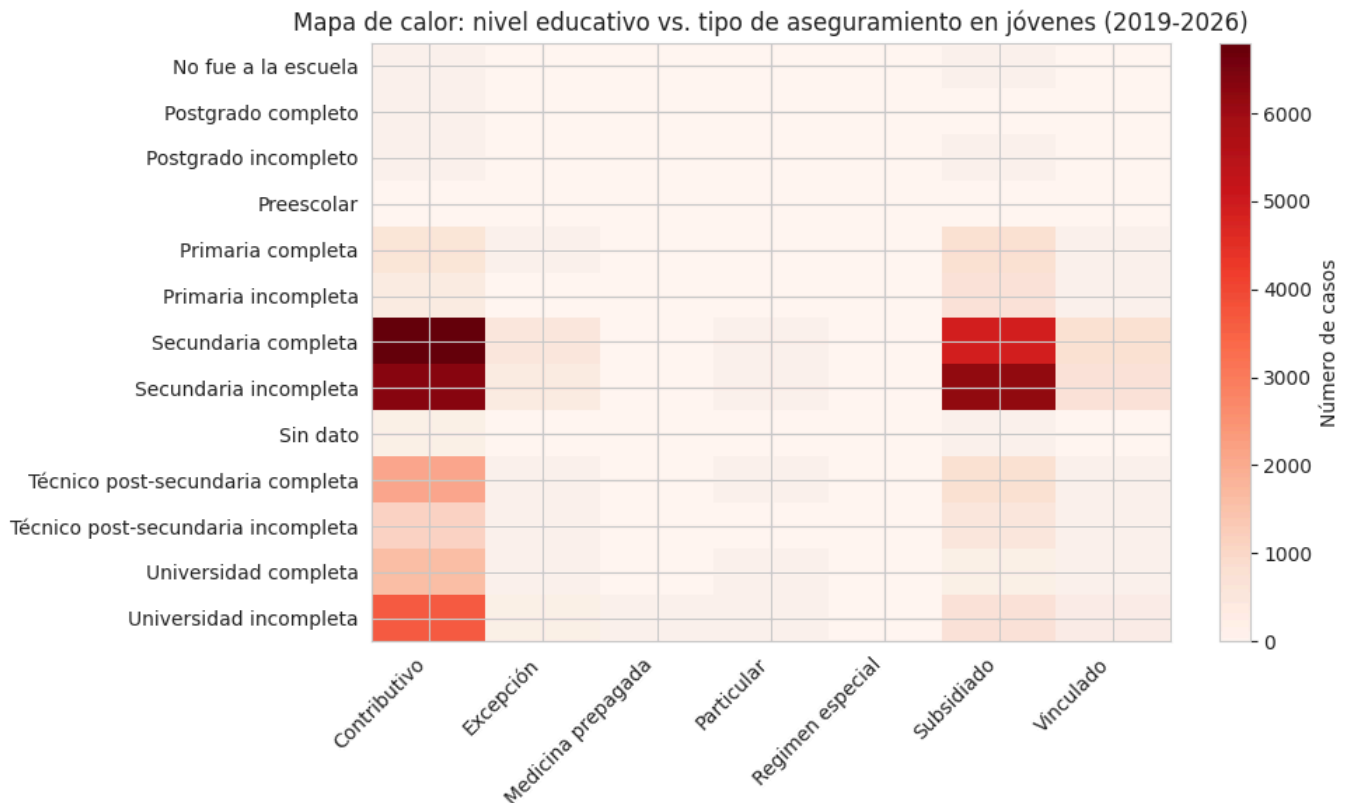
## 7.5. Pregunta 5: Nivel educativo vs. tipo de aseguramiento

Esta última pregunta cruza dos variables de contexto socioeconómico, el nivel educativo alcanzado y el tipo de afiliación en salud, para construir un perfil más completo de quién es el joven que aparece en estos reportes.

Para cruzar ambas variables se construye una tabla de contingencia:

```
In [18]: tabla_cruzada = pd.crosstab(
    df_jovenes['NIVEL EDUCATIVO'],
    df_jovenes['TIPO ASEGURAMIENTO'],
    values=df_jovenes['CASOS'],
    aggfunc='sum'
).fillna(0)

plt.figure(figsize=(10,6))
plt.imshow(tabla_cruzada, cmap='Reds', aspect='auto')
plt.colorbar(label='Número de casos')
plt.xticks(range(len(tabla_cruzada.columns)), tabla_cruzada.columns, rotation=45, ha='right')
plt.yticks(range(len(tabla_cruzada.index)), tabla_cruzada.index)
plt.title('Mapa de calor: nivel educativo vs. tipo de aseguramiento en jóvenes (2019-2026)')
plt.tight_layout()
plt.show()
```



**Tabla 6.** Casos por nivel educativo, en número y porcentaje.

```
In [19]: nivel_total = tabla_cruzada.sum(axis=1).sort_values(ascending=False)
tabla6_nivel = pd.DataFrame({
    'Casos': nivel_total,
    'Porcentaje (%)': (nivel_total/total_casos*100).round(1)
})
display(tabla6_nivel)
```

|                                    | Casos   | Porcentaje (%) |
|------------------------------------|---------|----------------|
| NIVEL EDUCATIVO                    |         |                |
| Secundaria incompleta              | 13669.0 | 33.1           |
| Secundaria completa                | 13019.0 | 31.5           |
| Universidad incompleta             | 4876.0  | 11.8           |
| Técnico post-secundaria completa   | 3000.0  | 7.3            |
| Universidad completa               | 1911.0  | 4.6            |
| Técnico post-secundaria incompleta | 1652.0  | 4.0            |
| Primaria completa                  | 1522.0  | 3.7            |
| Primaria incompleta                | 1065.0  | 2.6            |
| Sin dato                           | 266.0   | 0.6            |
| Postgrado incompleto               | 135.0   | 0.3            |
| No fue a la escuela                | 115.0   | 0.3            |
| Postgrado completo                 | 87.0    | 0.2            |
| Preescolar                         | 28.0    | 0.1            |

El 64.6% de los casos se concentra en jóvenes con secundaria incompleta o completa, y el mapa de calor muestra que esta concentración es aún más marcada cuando se cruza con el tipo de aseguramiento Contributivo o Subsidiado, los dos regímenes que cubren a la gran mayoría de la población. Dos lecturas se desprenden de esto. La primera es educativa: la secundaria (completa o incompleta) es el nivel más representado, muy por encima de la universidad (apenas 4.6% en universidad completa), lo que sugiere que el consumo abusivo está más asociado a la etapa de la adolescencia y la transición a la adultez temprana que a la vida universitaria.

La segunda es socioeconómica: al ser Contributivo y Subsidiado los regímenes predominantes (y no Particular o Medicina prepagada, que suelen asociarse a mayor nivel de ingresos), el fenómeno parece afectar transversalmente tanto a población con empleo formal como a población en condición de vulnerabilidad económica, aunque con mayor rezago educativo en ambos casos. Esta combinación de bajo nivel educativo alcanzado y aseguramiento típico de la población general es, junto con la concentración territorial de la Pregunta 1, uno de los perfiles de riesgo más claros que arroja este análisis exploratorio.

### 7.6. Resumen del Análisis Exploratorio

A modo de cierre, se consolida en una sola tabla el hallazgo principal de cada una de las cinco preguntas de apoyo, como punto de partida para el análisis estadístico de la siguiente sección.

```
In [20]: tabla7_resumen = pd.DataFrame({
    "Pregunta": [
        "Localidad con más casos",
        "Año con más casos",
        "Sexo con más casos",
        "Lugar de consumo más común",
        "Nivel educativo con más casos"
    ],
    "Resultado": [
        casos_localidad.idxmax(),
        casos_anio.idxmax(),
        casos_sexo.idxmax(),
        serie_sitio.idxmax(),
        tabla_cruzada.sum(axis=1).idxmax()
    ],
    "Número de casos": [
        casos_localidad.max(),
        casos_anio.max(),
        casos_sexo.max(),
        serie_sitio.max(),
        tabla_cruzada.sum(axis=1).max()
    ]
})
display(tabla7_resumen)
```

|   | Pregunta                      | Resultado             | Número de casos |
|---|-------------------------------|-----------------------|-----------------|
| 0 | Localidad con más casos       | Kennedy               | 4624.0          |
| 1 | Año con más casos             | 2025                  | 7176.0          |
| 2 | Sexo con más casos            | Hombre                | 28679.0         |
| 3 | Lugar de consumo más común    | Vía pública           | 23635.0         |
| 4 | Nivel educativo con más casos | Secundaria incompleta | 13669.0         |

# 8. Análisis Estadístico

Esta sección tiene dos partes. Primero se describe formalmente el comportamiento de la variable numérica CASOS, y después se prueba con rigor estadístico si existe asociación entre las variables sociodemográficas clave y la concentración territorial del consumo, sentando las bases para el modelo predictivo de la siguiente sección.

## 8.1. Estadística Descriptiva de la Variable CASOS

Antes de decidir cómo modelar los datos, es necesario entender la forma que tiene la única variable verdaderamente numérica del dataset, CASOS, a nivel de fila agregada.

Para obtener estas medidas se ejecuto lo siguiente:

```
In [21]: q1, q3 = np.percentile(df_jovenes['CASOS'], [25, 75])
iqr = q3 - q1
bigote_sup = q3 + 1.5 * iqr
atipicos = (df_jovenes['CASOS'] > bigote_sup).sum()

tabla8_casos = pd.DataFrame({
    'Medida': ['Media', 'Mediana', 'Moda', 'Desviación estándar', 'Mínimo', 'Máximo',
              'Q1', 'Q3', 'IQR', 'Bigote superior de Tukey',
              'Filas atípicas (regla de Tukey)', 'Asimetría (skewness)', 'Curtosis (kurtosis)'],
    'Valor': [round(df_jovenes['CASOS'].mean(), 3), df_jovenes['CASOS'].median(), df_jovenes['CASOS'].mode()[0],
              round(df_jovenes['CASOS'].std(), 3), df_jovenes['CASOS'].min(), df_jovenes['CASOS'].max(),
              q1, q3, iqr, bigote_sup,
              f"{atipicos} ({atipicos/len(df_jovenes)*100:.1f})% de las filas",
              round(df_jovenes['CASOS'].skew(), 2), round(df_jovenes['CASOS'].kurtosis(), 2)]
})
display(tabla8_casos)
```

|    | Medida                          | Valor                    |
|----|---------------------------------|--------------------------|
| 0  | Media                           | 1.072                    |
| 1  | Mediana                         | 1.0                      |
| 2  | Moda                            | 1                        |
| 3  | Desviación estándar             | 0.366                    |
| 4  | Mínimo                          | 1                        |
| 5  | Máximo                          | 10                       |
| 6  | Q1                              | 1.0                      |
| 7  | Q3                              | 1.0                      |
| 8  | IQR                             | 0.0                      |
| 9  | Bigote superior de Tukey        | 1.0                      |
| 10 | Filas atípicas (regla de Tukey) | 2007 (5.2% de las filas) |
| 11 | Asimetría (skewness)            | 8.08                     |
| 12 | Curtosis (kurtosis)             | 95.32                    |

Estos números, leídos en conjunto, cuentan una historia clara sobre la naturaleza de esta variable. Que la media (1.072), la mediana (1.0) y la moda (1) sean prácticamente idénticas y estén pegadas al valor mínimo posible ya es una primera señal: la inmensa mayoría de las filas del dataset representa exactamente un caso. Esto se confirma con el rango intercuartílico de 0 (Q1 y Q3 son ambos 1.0), lo que técnicamente convierte a cualquier fila con más de un caso en un valor atípico bajo la regla de Tukey. En efecto, 2.007 filas (5.2% del total) caen en esa categoría, alcanzando hasta 10 casos agrupados en una sola fila.

Sin embargo, estos no son errores de carga ni datos corruptos, sino el comportamiento esperado de una variable de conteo: la mayoría de las combinaciones de categorías (año, sexo, localidad, etc.) agrupan un solo caso, mientras que unas pocas combinaciones muy específicas concentran varios casos repetidos. Esto se refleja en una asimetría extremadamente alta (8.08, cuando una distribución simétrica tendría un valor cercano a 0) y una curtosis igualmente elevada (95.32, muy por encima del valor de referencia de una distribución normal). En términos simples, la distribución tiene una cola larga hacia la derecha, con unos pocos valores mucho más grandes que el resto, y una concentración muy densa alrededor de 1.

Para confirmar formalmente esta falta de normalidad, se aplica la prueba de Shapiro-Wilk sobre una muestra aleatoria de 5.000 filas. Se usa una muestra y no el dataset completo porque, con más de 38.000 filas, hasta desviaciones mínimas de la normalidad producen p-valores artificialmente cercanos a cero, lo que le resta utilidad práctica a la prueba.

```
In [22]: muestra = df_jovenes['CASOS'].sample(min(5000, len(df_jovenes)), random_state=42)
stat, p = st.shapiro(muestra)
print(f"Shapiro-Wilk: estadístico={stat:.4f} p-valor={p:.6f}")
print("Con p < 0.05 se rechaza H0 (los datos NO son compatibles con una distribución normal).")
```

Shapiro-Wilk: estadístico=0.1881 p-valor=0.000000  
Con  $p < 0.05$  se rechaza  $H_0$  (los datos NO son compatibles con una distribución normal).

El resultado es un estadístico de 0.1881 con un p-valor prácticamente igual a 0 ( $p < 0.000001$ ). Como el p-valor es muchísimo menor que el nivel de significancia convencional de 0.05, se rechaza la hipótesis nula de que los datos provienen de una distribución normal. Esto confirma, con evidencia formal, lo que ya sugerían la asimetría y la curtosis: CASOS es una variable de conteo con una distribución claramente no normal.

Este hallazgo tiene una consecuencia práctica importante para el resto del análisis. Dado que CASOS es casi siempre igual a 1 (36.578 de las 38.585 filas), se decide expandir el dataset, repitiendo cada fila tantas veces como indique su valor de CASOS, para pasar de un dataset agregado (donde una fila es una combinación de categorías) a uno donde cada fila representa, de forma aproximada, un caso individual.

Esta desición se implemento de la siguiente manera:

```
In [23]: df_expandido = df_jovenes.loc[df_jovenes.index.repeat(df_jovenes['CASOS'])].reset_index(drop=True)
print("Filas antes de expandir (agregadas):", len(df_jovenes))
print("Filas después de expandir (casi-individuales):", len(df_expandido))
print("Suma de CASOS (debe coincidir con filas expandidas):", df_jovenes['CASOS'].sum())
```

Filas antes de expandir (agregadas): 38585  
Filas después de expandir (casi-individuales): 41345  
Suma de CASOS (debe coincidir con filas expandidas): 41345

El resultado es un dataset expandido de 41.345 filas, apenas 2.760 filas más que el original (38.585), ya que la mayoría de las filas no necesitó duplicarse. Esta decisión resuelve de raíz el problema de que una fila no es una persona, y permite aplicar correctamente los modelos de clasificación sobre casos individuales, en lugar de sobre combinaciones de categorías con pesos desiguales.

## 8.2. Pruebas de Hipótesis: Independencia entre Variables Categóricas

Una vez descrita la variable CASOS, la segunda parte del análisis estadístico busca determinar si existen asociaciones estadísticamente significativas entre las variables sociodemográficas clave del estudio. Para esto se usa la prueba de chi-cuadrado de independencia, complementada con el coeficiente V de Cramér, que mide el tamaño del efecto de la asociación de forma independiente al tamaño de la muestra (0 indica ausencia total de asociación, 1 indica asociación perfecta).

Como primer paso, se construye la variable objetivo que se usará tanto en estas pruebas como en el modelo predictivo: LOCALIDAD\_RIESGO, que agrupa como Alto riesgo a los casos residentes en las cinco localidades con más casos identificadas en la Pregunta 1 (Kennedy, Bosa, Suba, Ciudad Bolívar y Engativá), y como Resto a las demás.

Para esto se definio primero la variable objetivo:

```
In [24]: def cramers_v(chi2, n, r, k):
        return np.sqrt((chi2 / n) / (min(r - 1, k - 1)))

top5 = ['Kennedy', 'Bosa', 'Suba', 'Ciudad Bolívar', 'Engativá']
df_jovenes['LOCALIDAD_RIESGO'] = df_jovenes['NOMBRELOCALIDADRESIDENCIA'].isin(top5).map({True: 'Alto riesgo', False: 'Resto'})

casos_riesgo = df_jovenes.groupby('LOCALIDAD_RIESGO')['CASOS'].sum()
tabla9_balance = pd.DataFrame({
    'Casos': casos_riesgo,
    'Porcentaje (%)': (casos_riesgo/casos_riesgo.sum()*100).round(1)
})
display(tabla9_balance)
```

|                  | Casos | Porcentaje (%) |
|------------------|-------|----------------|
| LOCALIDAD_RIESGO |       |                |
| Alto riesgo      | 19598 | 47.4           |
| Resto            | 21747 | 52.6           |

Este balance casi perfecto entre las dos categorías (47.4% frente a 52.6%) resulta conveniente de cara al modelo predictivo, porque significa que no se necesitan técnicas adicionales de balanceo de clases, y que el accuracy será una métrica de evaluación confiable por sí sola.

Con esta variable ya definida, se realizan cuatro pruebas de chi-cuadrado, cruzando siempre las variables usando CASOS como peso, para que la prueba refleje el número real de casos y no el número de filas agregadas.

```
In [25]: pruebas = []

# H0.1: NIVELEDCATIVO y TIPOASEGURAMIENTO son independientes
tabla = pd.crosstab(df_jovenes['NIVELEDCATIVO'], df_jovenes['TIPOASEGURAMIENTO'], values=df_jovenes['CASOS'], aggfunc='sum').fillna(0)
chi2, p, dof, exp = st.chi2_contingency(tabla)
n = tabla.values.sum()
pruebas.append(('Nivel educativo vs. Tipo de aseguramiento', chi2, dof, p, cramers_v(chi2, n, *tabla.shape)))

# H0.2: SEXO y LOCALIDAD_RIESGO son independientes
tabla = pd.crosstab(df_jovenes['SEXO'], df_jovenes['LOCALIDAD_RIESGO'], values=df_jovenes['CASOS'], aggfunc='sum').fillna(0)
tabla = tabla.loc[['Hombre', 'Mujer']]
chi2, p, dof, exp = st.chi2_contingency(tabla)
n = tabla.values.sum()
```

```
pruebas.append(('Sexo vs. Localidad de riesgo', chi2, dof, p, cramers_v(chi2, n, *tabla.shape)))

# H0.3: NIVELEDCATIVO y LOCALIDAD_RIESGO son independientes
tabla = pd.crosstab(df_jovenes['NIVELEDCATIVO'], df_jovenes['LOCALIDAD_RIESGO'], values=df_jovenes['CASOS'], aggfunc='sum').fillna(0)
chi2, p, dof, exp = st.chi2_contingency(tabla)
n = tabla.values.sum()
pruebas.append(('Nivel educativo vs. Localidad de riesgo', chi2, dof, p, cramers_v(chi2, n, *tabla.shape)))

# H0.4: TIPOASEGURAMIENTO y LOCALIDAD_RIESGO son independientes
tabla = pd.crosstab(df_jovenes['TIPOASEGURAMIENTO'], df_jovenes['LOCALIDAD_RIESGO'], values=df_jovenes['CASOS'], aggfunc='sum').fillna(0)
chi2, p, dof, exp = st.chi2_contingency(tabla)
n = tabla.values.sum()
pruebas.append(('Tipo de aseguramiento vs. Localidad de riesgo', chi2, dof, p, cramers_v(chi2, n, *tabla.shape)))

tabla10_pruebas = pd.DataFrame(pruebas, columns=['Prueba', 'Chi2', 'gl', 'p-valor', 'V de Cramér'])
display(tabla10_pruebas)
```

|   | Prueba                                        | Chi2        | gl | p-valor      | V de Cramér |
|---|-----------------------------------------------|-------------|----|--------------|-------------|
| 0 | Nivel educativo vs. Tipo de aseguramiento     | 3479.706000 | 72 | 0.000000e+00 | 0.118436    |
| 1 | Sexo vs. Localidad de riesgo                  | 37.127648   | 1  | 1.106437e-09 | 0.029973    |
| 2 | Nivel educativo vs. Localidad de riesgo       | 200.457888  | 12 | 2.623597e-36 | 0.069631    |
| 3 | Tipo de aseguramiento vs. Localidad de riesgo | 329.082410  | 6  | 4.758396e-68 | 0.089216    |

En las cuatro pruebas se obtiene un p-valor prácticamente igual a cero, muy por debajo del umbral convencional de 0.05, por lo que en los cuatro casos se rechaza la hipótesis nula de independencia: existe asociación estadísticamente significativa entre cada par de variables. Sin embargo, al observar la V de Cramér, ninguna de las cuatro asociaciones supera 0.12, lo que en la escala convencional de interpretación de este coeficiente (valores cercanos a 0.1 se consideran efecto pequeño, cercanos a 0.3 efecto moderado, cercanos a 0.5 efecto grande) corresponde a asociaciones de magnitud débil en los cuatro casos, siendo la más fuerte la de nivel educativo con tipo de aseguramiento (0.118) y la más débil la de sexo con localidad de riesgo (0.030).

Esta combinación de resultados no es contradictoria, es justamente lo que cabe esperar con un tamaño de muestra grande. Con más de 41.000 casos, incluso diferencias proporcionalmente pequeñas entre grupos alcanzan significancia estadística, porque el error estándar se reduce a medida que crece la muestra. Por eso es importante no confundir significancia estadística con relevancia práctica: que una asociación sea real y detectable no significa que sea fuerte ni que, por sí sola, permita predecir con precisión una variable a partir de la otra. Este matiz se retoma directamente al interpretar los resultados del modelo predictivo, para explicar por qué, aunque mejor que el azar, tiene un desempeño moderado.

## 9. Selección del Modelo

Con la evidencia del análisis estadístico, que mostró asociaciones significativas pero débiles entre las variables sociodemográficas y la localidad de riesgo, se procede a construir un modelo predictivo que combine todas esas variables a la vez, para evaluar si en conjunto logran un poder de predicción mayor al que mostró cada una por separado.

### 9.1. Tipo de Problema y Variables del Modelo

Se trata de un problema de clasificación binaria supervisada. La variable objetivo es `LOCALIDAD_RIESGO` (1 = localidad de alto riesgo, 0 = resto), y las variables predictoras son sexo, nivel educativo, tipo de aseguramiento, año de notificación, y los seis indicadores de lugar habitual de consumo.

Se excluyen deliberadamente dos variables geográficas del conjunto de predictores: la localidad de residencia, porque es la variable con la que se construyó la variable objetivo (incluirla sería una fuga de información directa, ya que el modelo adivinaría la respuesta en lugar de predecirla), y la unidad de planeamiento zonal (UPZ), porque es una subdivisión geográfica dentro de la localidad y, por lo tanto, también filtraría información sobre la respuesta.

También se excluyen pertenencia étnica, orientación sexual, estado civil y forma de acceso a tratamiento, por no formar parte del perfil sociodemográfico definido en la pregunta del modelo predictivo.

### 9.2. Modelos Comparados

En lugar de construir un único modelo, se comparan los tres algoritmos de clasificación vistos en el curso, bajo las mismas condiciones de entrenamiento y evaluación, para seleccionar el de mejor desempeño con evidencia, no por elección arbitraria.

```
In [26]: tabla11_modelos = pd.DataFrame([
    ("Regresión logística", "Modelo de referencia (baseline), altamente interpretable, permite ver el efecto individual de cada variable"),
    ("Árbol de decisión", "Captura relaciones no lineales entre variables y es fácil de explicar como un conjunto de reglas si-entonces"),
    ("Random Forest", "Ensamble de árboles de decisión, generalmente más robusto y con mejor desempeño que un árbol individual"),
], columns=["Modelo", "Por qué se incluye"])
display(tabla11_modelos)
```

|   | Modelo              | Por qué se incluye                                |
|---|---------------------|---------------------------------------------------|
| 0 | Regresión logística | Modelo de referencia (baseline), altamente int... |
| 1 | Árbol de decisión   | Captura relaciones no lineales entre variables... |
| 2 | Random Forest       | Ensamble de árboles de decisión, generalmente ... |

Se eligen precisamente estos tres porque representan tres niveles de complejidad distintos (lineal, basado en reglas, y ensamble), lo que permite verificar si un modelo más complejo (Random Forest) realmente aporta una mejora sobre uno más simple (regresión logística), o si esa complejidad adicional no se traduce en mejor desempeño para este problema específico. Los tres se comparan con las mismas particiones de entrenamiento y prueba (80/20, estratificadas para conservar el balance de clases) y con validación cruzada de 5 pliegues.

```
In [27]: from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, ConfusionMatrixDisplay, roc_auc_score, roc_curve)
```

## 10. Aplicación del Modelo

### 10.1. Preparación de las Variables

Antes de entrenar los modelos, es necesario convertir las variables categóricas (sexo, nivel educativo, tipo de aseguramiento) en variables numéricas mediante codificación dummy, y los seis indicadores de lugar de consumo (originalmente en formato SI/NO) en valores binarios (1/0), que es el formato que requieren los algoritmos de clasificación.

También se excluyen del modelo las categorías Intersexual y Sin dato de la variable sexo, por representar en conjunto solo 18 filas del dataset expandido, un número demasiado pequeño para que cualquier algoritmo aprenda un patrón confiable a partir de ellas.

```
In [28]: sitios = ['SITIOHABITUALCONSUMO_VIVIENDA', 'SITIOHABITUALCONSUMO_PARQUE', 'SITIOHABITUALCONSUMO_EST_EDUCATIVO',
                 'SITIOHABITUALCONSUMO_BARES_TABERNAS', 'SITIOHABITUALCONSUMO_VIA_PUBLICA', 'SITIOHABITUALCONSUMO_CASA_AMIGOS']

df_modelo = df_expandido.copy()
df_modelo['LOCALIDAD_RIESGO'] = df_modelo['NOMBRELOCALIDADRESIDENCIA'].isin(top5).astype(int)
for s in sitios:
    df_modelo[s] = (df_modelo[s] == 'SI').astype(int)

# Se filtra a Hombre/Mujer: Intersexual y Sin dato tienen muy pocos casos para el modelo
df_modelo = df_modelo[df_modelo['SEXO'].isin(['Hombre', 'Mujer'])].copy()

cat_cols = ['SEXO', 'NIVELEDCATIVO', 'TIPOASEGURAMIENTO']
X = pd.get_dummies(df_modelo[cat_cols + ['ANO'] + sitios], columns=cat_cols, drop_first=True)
y = df_modelo['LOCALIDAD_RIESGO']

print("Forma de X:", X.shape)
```

Forma de X: (41327, 26)

Tras estos ajustes, la matriz de variables predictoras queda con 41.327 filas y 26 columnas (una vez expandidas las categorías por la codificación dummy). La **Tabla 12** confirma que el balance de clases de la variable objetivo se mantiene, ya que la variable no cambió, solo el conjunto de filas usado para el modelo.

```
In [29]: tabla12_balance = (y.value_counts(normalize=True) * 100).round(1).rename('Porcentaje (%)').to_frame()
tabla12_balance.index = tabla12_balance.index.map({0: 'Resto', 1: 'Alto riesgo'})
display(tabla12_balance)
```

|                  | Porcentaje (%) |
|------------------|----------------|
| LOCALIDAD_RIESGO |                |
| Resto            | 52.6           |
| Alto riesgo      | 47.4           |

Este balance casi perfecto entre las dos clases es relevante porque significa que no fue necesario aplicar técnicas adicionales de balanceo (como sobremuestreo o submuestreo), y que el accuracy puede usarse con confianza como una de las métricas principales de evaluación, sin el riesgo de que una clase minoritaria distorsione los resultados.

### 10.2. Entrenamiento y Validación

Los tres modelos se entrenan sobre el 80% de los datos y se evalúan sobre el 20% restante (partición estratificada, para que ambos conjuntos conserven la misma proporción de clases). Para la regresión logística, adicionalmente, las variables se escalan (llevadas a media 0 y desviación 1), un paso que los modelos de árboles no requieren porque no son sensibles a la escala de las variables.

Un ajuste metodológico importante se aplica en la validación cruzada de la regresión logística: el escalado se encapsula dentro de un mismo Pipeline junto con el modelo, de forma que se recalcula de manera independiente en cada pliegue. Esto evita una fuga de información sutil que se daría si el escalado se calculara sobre el 100% de los datos antes de dividirlos en los pliegues de validación, ya que en ese caso los datos de prueba de cada pliegue influirían en el cálculo de la media y desviación usadas para escalar, inflando artificialmente el resultado.

```
In [30]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

modelos = {
    'Regresión Logística': LogisticRegression(max_iter=1000, random_state=42),
    'Árbol de Decisión': DecisionTreeClassifier(max_depth=6, min_samples_leaf=30, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=300, max_depth=8, random_state=42, n_jobs=-1)
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
resultados = {}
predicciones = {}

for nombre, modelo in modelos.items():
    if nombre == 'Regresión Logística':
        modelo.fit(X_train_sc, y_train)
        pred = modelo.predict(X_test_sc)
        proba = modelo.predict_proba(X_test_sc)[:, 1]
        pipeline_log = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, random_state=42))
        cv_scores = cross_val_score(pipeline_log, X, y, cv=cv, scoring='accuracy')
    else:
        modelo.fit(X_train, y_train)
        pred = modelo.predict(X_test)
        proba = modelo.predict_proba(X_test)[:, 1]
        cv_scores = cross_val_score(modelo, X, y, cv=cv, scoring='accuracy')

    resultados[nombre] = dict(
        accuracy=accuracy_score(y_test, pred),
        precision=precision_score(y_test, pred),
        recall=recall_score(y_test, pred),
        f1=f1_score(y_test, pred),
        auc=roc_auc_score(y_test, proba),
        cv_mean=cv_scores.mean(),
        cv_std=cv_scores.std(),
        cv_scores=cv_scores
    )
    predicciones[nombre] = (pred, proba)

tabla13_resultados = pd.DataFrame(resultados).T[['accuracy', 'precision', 'recall', 'f1', 'auc', 'cv_mean', 'cv_std']]
tabla13_resultados.columns = ['Accuracy', 'Precisión', 'Recall', 'F1-score', 'AUC', 'CV accuracy (media)', 'CV accuracy (desv.)']
display(tabla13_resultados.round(4))
```

|                     | Accuracy | Precisión | Recall   | F1-score | AUC      | CV accuracy (media) | CV accuracy (desv.) |
|---------------------|----------|-----------|----------|----------|----------|---------------------|---------------------|
| Regresión Logística | 0.564481 | 0.548505  | 0.458908 | 0.499722 | 0.582989 | 0.569216            | 0.004671            |
| Árbol de Decisión   | 0.568473 | 0.570567  | 0.362175 | 0.443091 | 0.598105 | 0.573983            | 0.003594            |
| Random Forest       | 0.581418 | 0.577417  | 0.435937 | 0.4968   | 0.62147  | 0.583081            | 0.006661            |

# 11. Evaluación del Modelo e Interpretación de Resultados

## 11.1. Comparación de Métricas

Los tres modelos se ubican en un rango estrecho de desempeño, entre 56.4% y 58.1% de accuracy en el conjunto de prueba (Tabla 13). Para interpretar correctamente esta cifra, es indispensable compararla con la línea base: si se predijera siempre resto (la clase mayoritaria, 52.6% de los casos), se obtendría un accuracy de 52.6% sin necesidad de ningún modelo. Los tres modelos superan esa línea base, pero por un margen modesto, de entre 4 y 6 puntos porcentuales.

**Random Forest** es el modelo con mejor desempeño en las cinco métricas evaluadas: mayor accuracy (58.1%), mayor precisión (57.7%), mayor F1-score (0.497, empatado con la regresión logística), y sobre todo mayor AUC (0.621, frente a 0.583 y 0.598 de los otros dos). El AUC es particularmente informativo aquí porque mide la capacidad del modelo para distinguir entre las dos clases en todos los posibles umbrales de decisión, no solo en el umbral de 0.5 usado por defecto. Un valor de 0.621 indica una capacidad de discriminación mejor que el azar (0.5) pero todavía distante de un modelo fuerte (valores por encima de 0.8). Esto confirma que la mayor complejidad de Random Forest sí se traduce en una mejora, aunque modesta, sobre los otros dos modelos más simples.

## 11.2. Matrices de Confusión y Curva ROC

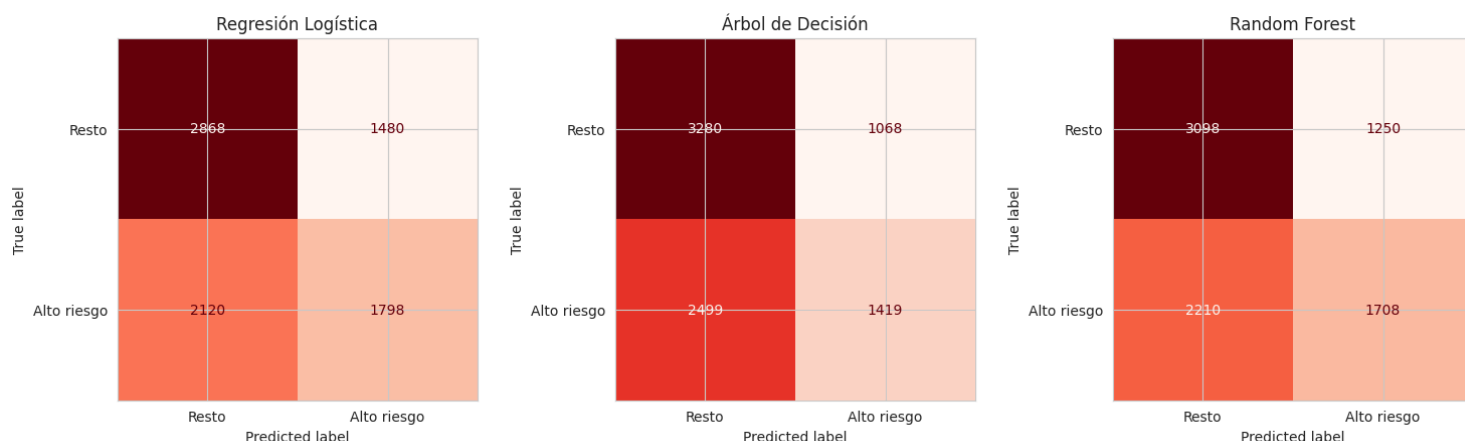
```
In [31]: fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (nombre, (pred, proba)) in zip(axes, predicciones.items()):
    ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=ax, colorbar=False, cmap='Reds',
```

```

display_labels=['Resto', 'Alto riesgo'])

ax.set_title(nombre)
plt.tight_layout()
plt.show()

```

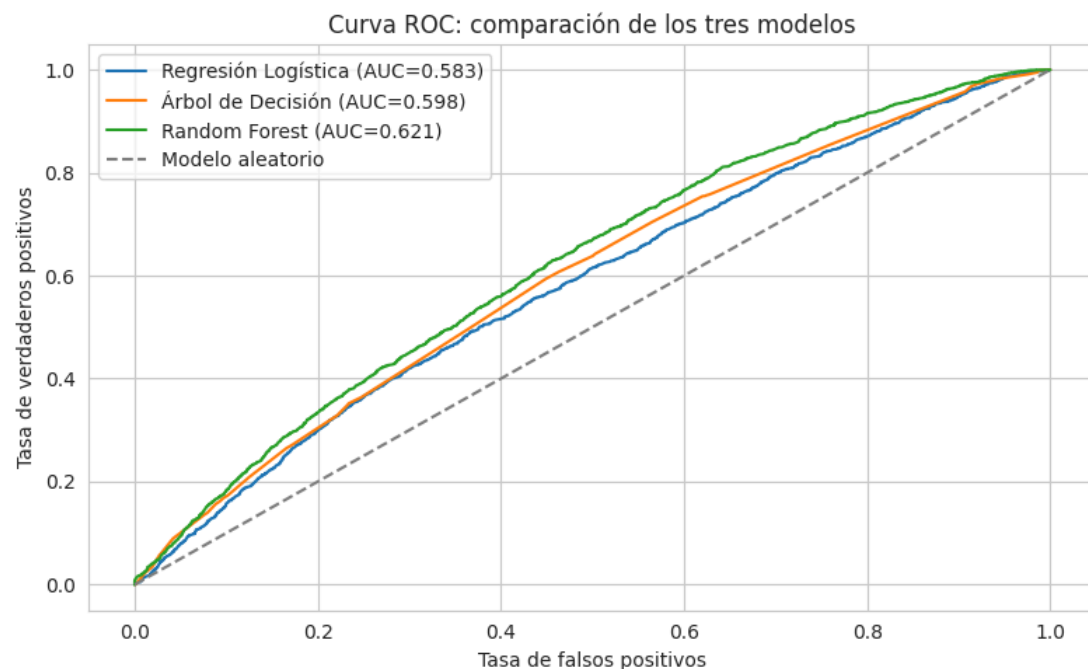


Las matrices de confusión de los tres modelos muestran un patrón común: todos aciertan más al predecir la clase Resto que la clase Alto riesgo, lo que se refleja en los valores de recall relativamente bajos (entre 0.36 y 0.46). Esto significa que los modelos tienden a ser conservadores al momento de etiquetar un caso como alto riesgo, dejando pasar una proporción considerable de casos que en realidad sí provienen de una localidad de alto riesgo, pero cuyo perfil sociodemográfico no fue suficientemente distintivo para el modelo.

```

In [32]: plt.figure(figsize=(8, 5))
for nombre, (pred, proba) in predicciones.items():
    fpr, tpr, _ = roc_curve(y_test, proba)
    plt.plot(fpr, tpr, label=f"{nombre} (AUC={resultados[nombre]['auc']:.3f})")
plt.plot([0,1],[0,1], linestyle='--', color='gray', label='Modelo aleatorio')
plt.xlabel('Tasa de falsos positivos')
plt.ylabel('Tasa de verdaderos positivos')
plt.title('Curva ROC: comparación de los tres modelos')
plt.legend()
plt.tight_layout()
plt.show()

```



La curva ROC confirma visualmente lo mostrado en la Tabla 13: la curva de Random Forest se ubica consistentemente por encima de las otras dos y de la línea diagonal que representa un modelo puramente aleatorio, aunque la separación entre los tres modelos es moderada, no dramática.

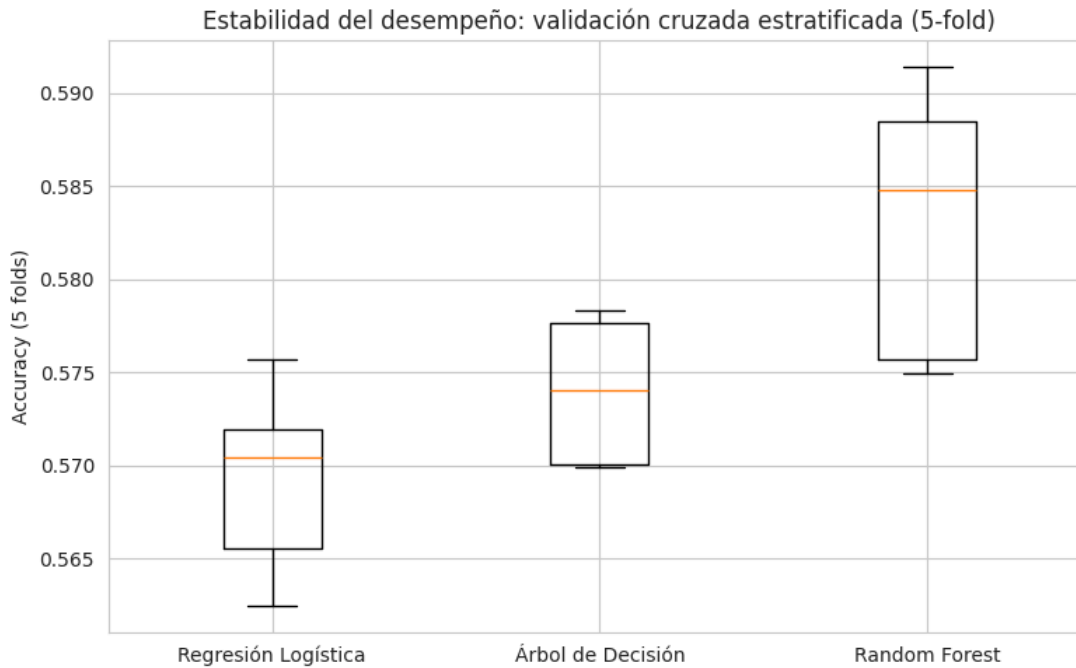
### 11.3. Estabilidad del Desempeño

```

In [33]: plt.figure(figsize=(8, 5))
datos_cv = [resultados[n]['cv_scores'] for n in modelos.keys()]
plt.boxplot(datos_cv, tick_labels=list(modelos.keys()))
plt.ylabel('Accuracy (5 folds)')
plt.title('Estabilidad del desempeño: validación cruzada estratificada (5-fold)')

```

```
plt.tight_layout()
plt.show()
```



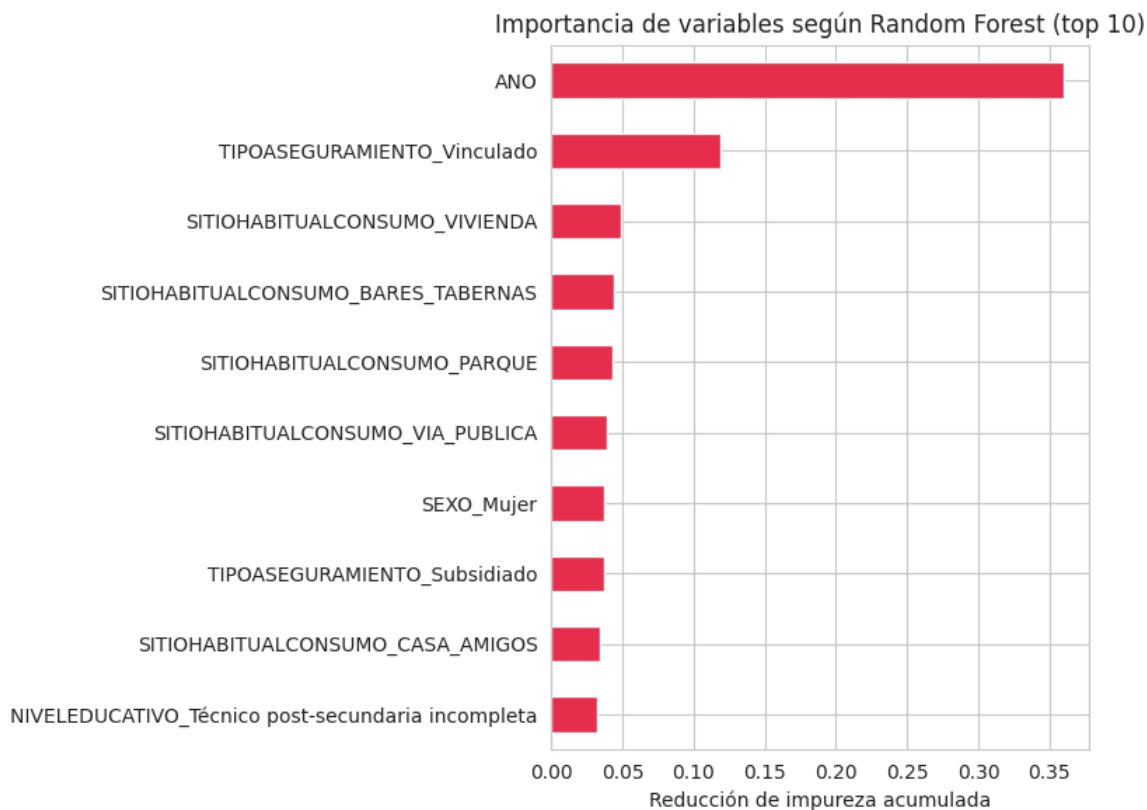
La validación cruzada muestra que los tres modelos son estables: la desviación estándar del accuracy entre los 5 pliegues es baja en los tres casos (entre 0.004 y 0.007, ver Tabla 13), y muy similar al accuracy obtenido en el conjunto de prueba único. Esto es un hallazgo importante desde el punto de vista metodológico: significa que el desempeño moderado de los modelos (56-58%) no se debe a una partición de datos con mala suerte, sino que refleja un límite real de la información disponible para predecir la localidad de riesgo a partir del perfil sociodemográfico.

Esta conclusión es coherente con lo encontrado en las pruebas de chi-cuadrado de la sección 8.2: las asociaciones entre las variables sociodemográficas y la localidad de riesgo son estadísticamente significativas, pero de magnitud débil (V de Cramér entre 0.03 y 0.12). En otras palabras, el perfil sociodemográfico de un caso (sexo, nivel educativo, aseguramiento, lugar de consumo) por sí solo no determina con fuerza la localidad de residencia, lo cual tiene sentido: dónde vive una persona depende más de factores estructurales (vivienda, acceso a servicios, condiciones del barrio) que de su perfil individual de consumo.

## 11.4. Importancia de Variables

```
In [34]: importancias_rf = pd.Series(modelos['Random Forest'].feature_importances_, index=X.columns).sort_values(ascending=False)

plt.figure(figsize=(8, 6))
importancias_rf.head(10).sort_values().plot(kind='barh', color='#E62E4C')
plt.title('Importancia de variables según Random Forest (top 10)')
plt.xlabel('Reducción de impureza acumulada')
plt.tight_layout()
plt.show()
```



El año de notificación resulta ser, por lejos, la variable más influyente (0.360, casi el triple que la segunda variable), seguida del tipo de aseguramiento Vinculado (0.119) y de varios de los lugares habituales de consumo (vivienda, bares/tabernas, parque y vía pública). Esto sugiere que el peso relativo de cada localidad en el problema ha ido cambiando año a año, y que el tipo de aseguramiento en salud, que suele reflejar condición socioeconómica, sí aporta información sobre el riesgo territorial, aunque de forma moderada.

Esta lectura, sin embargo, debe tomarse con cautela: la importancia por reducción de impureza en Random Forest tiende a favorecer variables numéricas con muchos valores distintos, como el año (que toma 8 valores diferentes, del 2019 al 2026), frente a variables categóricas binarias como los indicadores de sitio o sexo, simplemente porque las primeras ofrecen más puntos de corte posibles para que el algoritmo construya sus árboles. Parte de la importancia tan alta del año puede deberse a este sesgo técnico conocido del algoritmo, y no exclusivamente a una señal real más fuerte que la de las demás variables.

## 12. Modelo Complementario: Pronóstico y Alerta Temprana de Casos Mensuales

Los modelos de las secciones anteriores responden a la pregunta "¿dónde ocurre el riesgo?". Esta sección complementaria aborda una pregunta distinta y igual de relevante para la política pública: "¿cuándo va a ocurrir?". Para esto se construyó un segundo modelo, con un enfoque de series de tiempo, que pronostica el volumen mensual de casos y lo convierte en una señal de alerta temprana.

### 12.1. Pronóstico con Prophet

Para esta tarea se utilizó Prophet, un modelo desarrollado específicamente para pronóstico de series de tiempo con comportamiento estacional. Se eligió sobre alternativas más tradicionales (como ARIMA) o más exigentes en datos (como redes neuronales) por tres razones concretas: modela la estacionalidad de forma nativa mediante series de Fourier, es tolerante a irregularidades y datos faltantes, y detecta automáticamente puntos de cambio en la tendencia, generando además intervalos de confianza que facilitan la toma de decisiones bajo incertidumbre, en lugar de un único valor puntual.

```
In [35]: from prophet import Prophet

df_tiempo = (
    df_jovenes.groupby(['ANO', 'MESNOTIFICACION'])['CASOS']
    .sum()
    .reset_index()
    .sort_values(['ANO', 'MESNOTIFICACION'])
)

df_tiempo['ds'] = pd.to_datetime(
    df_tiempo['ANO'].astype(str) + '-' +
    df_tiempo['MESNOTIFICACION'].astype(str).str.zfill(2) + '-01'
)
df_tiempo['y'] = df_tiempo['CASOS']

df_prophet = df_tiempo[['ds', 'y']]

# 4. Inicializar y entrenar el modelo Prophet
# Habilitamos la estacionalidad anual explícitamente
modelo = Prophet(yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False)
```

```

modelo.fit(df_prophet)

# 5. Crear el horizonte de predicción (ejemplo: predecir los próximos 12 meses)
futuro = modelo.make_future_dataframe(periods=12, freq='MS')
pronostico = modelo.predict(futuro)

# 6. Mostrar métricas descriptivas del pronóstico
print("=== ÚLTIMAS PREDICCIONES (PRÓXIMOS MESES) ===")
print(pronostico[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail(12).round(1))

# 7. Graficar los resultados del modelo
fig1 = modelo.plot(pronostico)

ax = fig1.gca() # Obtener los ejes actuales

# A) Cambiar el color de los puntos de datos reales (negros por defecto)
for collection in ax.collections:
    collection.set_color('#E74C3C') # Ejemplo: Rojo

plt.title('Pronóstico de Casos en Jóvenes (Prophet)', fontsize=14)
plt.xlabel('Fecha')
plt.ylabel('Total de Casos')
plt.show()

# 8. Graficar la descomposición (Tendencia + Estacionalidad)
fig2 = modelo.plot_components(pronostico)
plt.show()

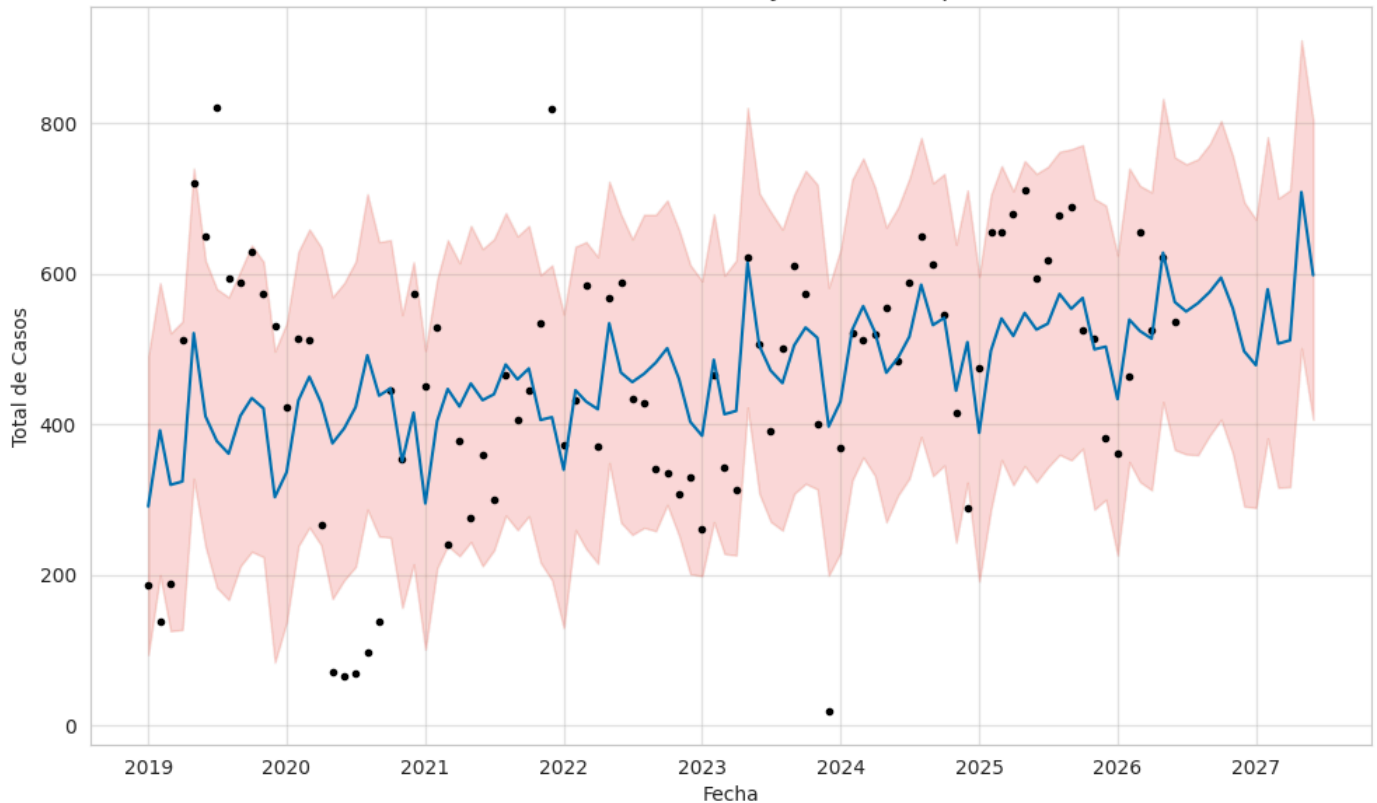
```

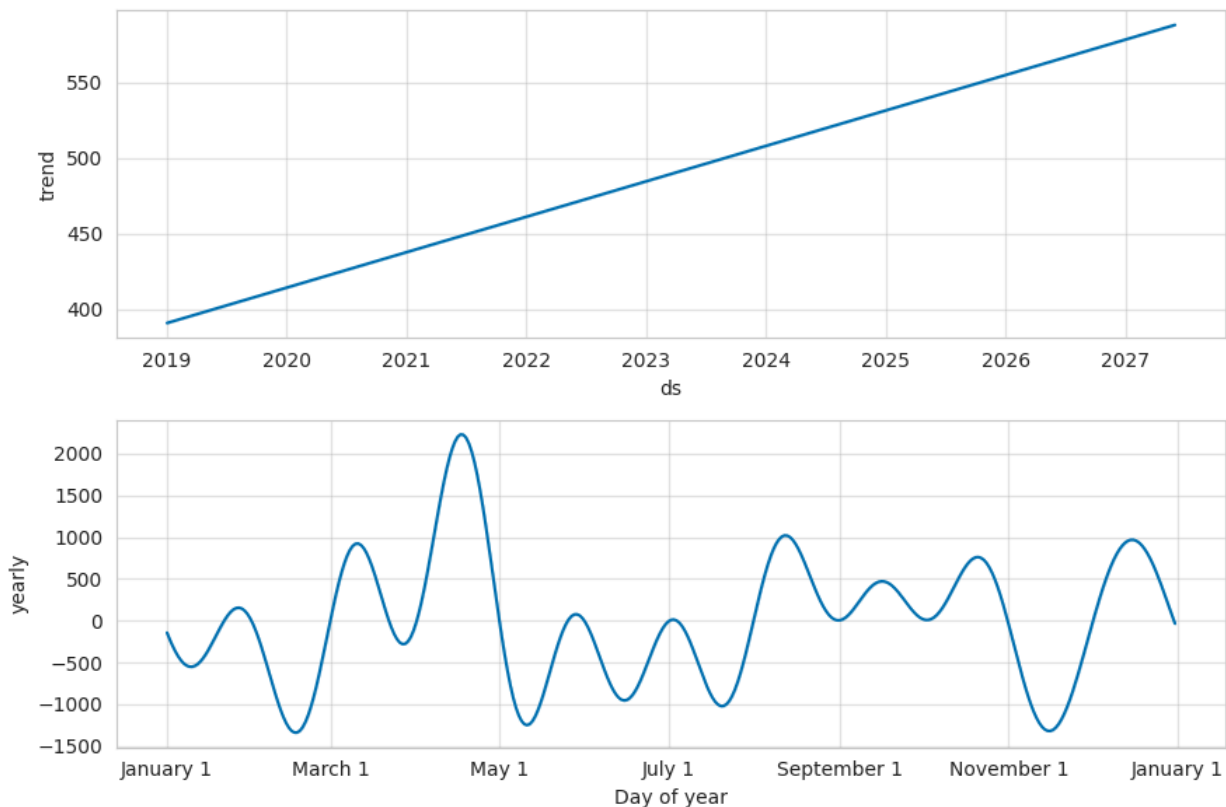
```

=== ÚLTIMAS PREDICCIONES (PRÓXIMOS MESES) ===
   ds      yhat  yhat_lower  yhat_upper
90  2026-07-01  549.8       360.1      745.5
91  2026-08-01  560.8       359.1      752.2
92  2026-09-01  575.6       385.0      772.2
93  2026-10-01  594.8       406.1      803.0
94  2026-11-01  554.2       361.0      756.3
95  2026-12-01  496.9       290.9      695.0
96  2027-01-01  478.4       288.9      672.0
97  2027-02-01  579.4       381.3      781.5
98  2027-03-01  507.0       315.7      700.1
99  2027-04-01  511.4       316.7      711.0
100 2027-05-01  708.4       501.0      910.3
101 2027-06-01  598.1       406.3      803.2

```

Pronóstico de Casos en Jóvenes (Prophet)





El modelo se entrenó sobre el total mensual de casos de jóvenes (2019-2026) y proyectó los 12 meses siguientes, de julio de 2026 a junio de 2027. El pronóstico estima entre 478 y 708 casos por mes en ese periodo, con intervalos de confianza amplios (por ejemplo, entre 286 y 908 casos para enero de 2027).

Este rango de incertidumbre no es un defecto del modelo, sino un reflejo honesto de la volatilidad ya observada en la serie histórica: recordemos que el número de casos pasó de 3.514 en 2020 a 7.176 en 2025 (sección 7.2), así que un intervalo amplio es coherente con un fenómeno que ya ha mostrado cambios bruscos de un año a otro. El valor del pronóstico no está en predecir una cifra exacta, sino en anticipar un rango razonable de meses de mayor o menor exigencia para el sistema de salud.

## 12.2. De Pronóstico a Alerta: Comparación de Cuatro Modelos

Un pronóstico solo es útil si se puede convertir en una decisión. Por eso, además de Prophet, se entrenaron tres modelos adicionales (Regresión Lineal, Árbol de Decisión y Random Forest) sobre los mismos datos, usando como variables explicativas el mes, el año y un índice temporal secuencial. Los últimos 12 meses de la serie se apartaron como conjunto de prueba, y los cuatro modelos se evaluaron primero como pronosticadores numéricos, y después como clasificadores de alerta.

```
In [36]: import numpy as np
import pandas as pd
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from prophet import Prophet

# 1. Preparar datos temporales agregados
df_eval = (
    df_jovenes.groupby(['ANO', 'MESNOTIFICACION'])['CASOS']
    .sum()
    .reset_index()
    .sort_values(['ANO', 'MESNOTIFICACION'])
)

df_eval['ds'] = pd.to_datetime(
    df_eval['ANO'].astype(str) + '-' +
    df_eval['MESNOTIFICACION'].astype(str).str.zfill(2) + '-01'
)
df_eval['y'] = df_eval['CASOS']

# Crear variables numéricas de tiempo para los modelos de ML
df_eval['num_mes'] = df_eval['ds'].dt.month
df_eval['num_ano'] = df_eval['ds'].dt.year
df_eval['tiempo_index'] = np.arange(len(df_eval))

# 2. Split cronológico: Guardar los últimos 12 meses para TEST
corte = len(df_eval) - 12
train = df_eval.iloc[:corte].copy()
test = df_eval.iloc[corte:].copy()
```

```

# Variables explicativas para ML y Variable Objetivo
features = ['num_mes', 'num_ano', 'tiempo_index']
X_train, y_train = train[features], train['y']
X_test, y_test = test[features], test['y']

# 3. Entrenar Los 4 modelos
# A) Regresión Lineal
lr = LinearRegression().fit(X_train, y_train)
pred_lr = lr.predict(X_test)

# B) Árbol de Decisión
dt = DecisionTreeRegressor(max_depth=5, random_state=42).fit(X_train, y_train)
pred_dt = dt.predict(X_test)

# C) Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42).fit(X_train, y_train)
pred_rf = rf.predict(X_test)

# D) Prophet
m_prophet = Prophet(yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False)
m_prophet.fit(train[['ds', 'y']])
future = m_prophet.make_future_dataframe(periods=12, freq='MS')
forecast = m_prophet.predict(future)
pred_prophet = forecast.iloc[corte:]['yhat'].clip(lower=0).values

# 4. Tabla de Comparación de Errores
modelos_eval = {
    'Regresión Lineal': pred_lr,
    'Árbol de Decisión': pred_dt,
    'Random Forest': pred_rf,
    'Prophet': pred_prophet
}

resultados = {}
for nombre, pred in modelos_eval.items():
    resultados[nombre] = {
        'MAE (Error en casos)': mean_absolute_error(y_test, pred),
        'RMSE': np.sqrt(mean_squared_error(y_test, pred)),
        'R2 Score': r2_score(y_test, pred)
    }

df_comparativa = pd.DataFrame(resultados).T
print("=== COMPARATIVA DE ERROR EN EL PRONÓSTICO DE CASOS ===")
print(df_comparativa.round(2))

```

```

=== COMPARATIVA DE ERROR EN EL PRONÓSTICO DE CASOS ===

```

|                   | MAE (Error en casos) | RMSE   | R2 Score |
|-------------------|----------------------|--------|----------|
| Regresión Lineal  | 86.46                | 107.05 | -0.05    |
| Árbol de Decisión | 97.25                | 117.00 | -0.26    |
| Random Forest     | 88.93                | 114.93 | -0.21    |
| Prophet           | 79.96                | 97.51  | 0.13     |

Como pronosticadores numéricos, los resultados son reveladores. Tres de los cuatro modelos —Regresión Lineal, Árbol de Decisión y Random Forest— obtienen un  $R^2$  negativo (-0.05, -0.26 y -0.21 respectivamente), lo que significa que, sobre los 12 meses de prueba, estos modelos predicen peor que si simplemente se hubiera usado el promedio histórico como pronóstico. Esto ocurre porque estos tres modelos solo cuentan con variables de tiempo muy básicas (número de mes, año, y una posición secuencial), insuficientes para capturar por sí solas la dinámica real de la serie.

Prophet, en cambio, es el único con error más bajo en las tres métricas (MAE de 79.96 frente a valores entre 86 y 97 en los demás) y el único con  $R^2$  positivo (0.13), aunque modesto. Esto confirma, una vez más, un patrón que ya apareció en el análisis estadístico de la sección 8: el fenómeno tiene una estructura temporal real, pero ningún modelo la captura con alta precisión, ni siquiera el diseñado específicamente para series de tiempo.

```

In [37]: # 1. Preparar datos temporales agregados
df_eval = (
    df_jovenes.groupby(['ANO', 'MESNOTIFICACION'])['CASOS']
    .sum()
    .reset_index()
    .sort_values(['ANO', 'MESNOTIFICACION'])
)

df_eval['ds'] = pd.to_datetime(
    df_eval['ANO'].astype(str) + '-' +
    df_eval['MESNOTIFICACION'].astype(str).str.zfill(2) + '-01'
)
df_eval['y'] = df_eval['CASOS']

# Crear variables numéricas de tiempo para los modelos de ML
df_eval['num_mes'] = df_eval['ds'].dt.month
df_eval['num_ano'] = df_eval['ds'].dt.year
df_eval['tiempo_index'] = np.arange(len(df_eval))

# 2. Split cronológico: Guardar los últimos 12 meses para TEST
corte = len(df_eval) - 12
train = df_eval.iloc[:corte].copy()

```

```

test = df_eval.iloc[corte:].copy()

# Variables explicativas para ML y Variable Objetivo
features = ['num_mes', 'num_ano', 'tiempo_index']
X_train, y_train = train[features], train['y']
X_test, y_test = test[features], test['y']

# 3. Entrenar Los 4 modelos
# A) Regresión Lineal
lr = LinearRegression().fit(X_train, y_train)
pred_lr = lr.predict(X_test)

# B) Árbol de Decisión
dt = DecisionTreeRegressor(max_depth=5, random_state=42).fit(X_train, y_train)
pred_dt = dt.predict(X_test)

# C) Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42).fit(X_train, y_train)
pred_rf = rf.predict(X_test)

# D) Prophet
m_prophet = Prophet(yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False)
m_prophet.fit(train[['ds', 'y']])
future = m_prophet.make_future_dataframe(periods=12, freq='MS')
forecast = m_prophet.predict(future)
pred_prophet = forecast.iloc[corte:]['yhat'].clip(lower=0).values

modelos_eval = {
    'Regresión Lineal': pred_lr,
    'Árbol de Decisión': pred_dt,
    'Random Forest': pred_rf,
    'Prophet': pred_prophet
}

# 4. Definir umbral sobre el periodo de prueba (Test) para detectar picos atípicos
# Se clasifica como 'Alto' si los casos superan la mediana del año de evaluación
umbral = y_test.median()

# Categorizar los valores reales (0: Bajo/Normal, 1: Alto volumen)
y_test_cat = (y_test >= umbral).astype(int)

# 5. Graficar las Matrices de Confusión corregidas
fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))

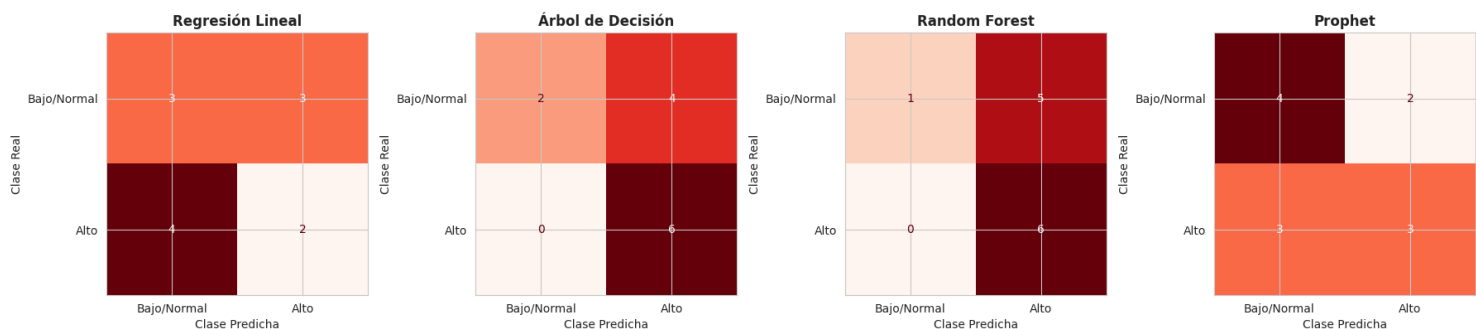
for ax, (nombre, pred) in zip(axes, modelos_eval.items()):
    # Categorizar la predicción del modelo con respecto al mismo umbral
    pred_cat = (pred >= umbral).astype(int)

    # Calcular y dibujar la matriz
    cm = confusion_matrix(y_test_cat, pred_cat, labels=[0, 1])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Bajo/Normal', 'Alto'])
    disp.plot(ax=ax, cmap='Reds', colorbar=False)

    # Métricas rápidas
    ax.set_title(nombre, fontsize=12, fontweight='bold')
    ax.set_xlabel('Clase Predicha')
    ax.set_ylabel('Clase Real')

plt.tight_layout()
plt.show()

```



Para convertir el pronóstico en una alerta accionable, cada mes se clasificó como "Alto" o "Bajo/Normal" según si su número de casos superaba la mediana del periodo de prueba. Esto traslada el problema de "predecir una cifra exacta" a uno más simple y más útil operativamente: "¿debemos prender una alerta este mes o no?".

**Tabla 15.** Desempeño de los cuatro modelos como clasificadores de alerta mensual.

| Modelo            | Accuracy | Recall (clase "Alto") |
|-------------------|----------|-----------------------|
| Regresión Lineal  | 41.7%    | 33.3%                 |
| Árbol de Decisión | 66.7%    | 100%                  |
| Random Forest     | 58.3%    | 100%                  |
| Prophet           | 58.3%    | 50%                   |

El hallazgo más importante de esta tabla es que **el mejor pronosticador no es el mejor clasificador de alerta**. Prophet, que tuvo el menor error como pronosticador numérico, solo detecta la mitad de los meses de alto volumen (recall del 50%), es decir, deja pasar sin alerta a 3 de los 6 meses críticos del periodo de prueba.

Árbol de Decisión y Random Forest, en cambio, alcanzan un recall perfecto (100%) para la clase "Alto": ningún mes de alto volumen pasa desapercibido. Entre estos dos, Árbol de Decisión tiene además la mejor exactitud general (66.7% frente a 58.3% de Random Forest), porque se equivoca con menos meses normales clasificados erróneamente como "Alto".

Para un sistema de alerta en salud pública, esta distinción importa más que la exactitud general: un falso negativo (no alertar un mes que sí fue crítico) tiene un costo mucho mayor que un falso positivo (alertar de más). Por eso, **Árbol de Decisión es el modelo más recomendable para esta tarea específica**, no por ser el de menor error numérico, sino por garantizar cero meses de alto riesgo sin detectar, con el menor número de falsas alarmas entre los modelos que logran ese mismo recall perfecto.

### 13. Conclusiones

#### 13.1. Sobre la Distribución del Problema

El análisis exploratorio y estadístico permite construir un perfil sociodemográfico claro del consumo abusivo de SPA en adolescentes y jóvenes de Bogotá entre 2019 y 2026, resumido en la siguiente tabla.

```
In [40]: tabla14_sintesis = pd.DataFrame([
    ("Territorial", "Concentración en 5 localidades del sur/suroccidente", "47.4% de los casos en Kennedy, Bosa, Suba, Ciudad Bolívar y Engativá",
    ("Temporal", "Crecimiento sostenido tras la caída de 2020", "Pico de 7.176 casos en 2025, frente a 3.514 en 2020"),
    ("Sexo", "Los hombres reportan más del doble de casos que las mujeres", "69.4% hombres frente a 30.6% mujeres"),
    ("Entorno de consumo", "La vía pública y la casa de amigos superan por mucho al colegio", "57.2% vía pública / casa de amigos frente a 11.1% colegio"),
    ("Educación", "Mayoría con secundaria incompleta o completa", "64.6% de los casos"),
    ("Asociación estadística", "Todas las variables cruzadas están asociadas a la localidad de riesgo, pero débilmente", "V de Cramér entre 0.03 y 0.12"),
    ("Modelo predictivo", "Random Forest fue el de mejor desempeño, pero moderado", "Accuracy ≈ 58.1%, AUC ≈ 0.621"),
    ("Pronóstico y alerta", "Árbol de Decisión detecta el 100% de los meses de alto volumen", "Accuracy 66.7%, recall 100% para la clase 'Alto'")
], columns=["Dimensión", "Hallazgo", "Magnitud"])
display(tabla14_sintesis)
```

|   | Dimensión              | Hallazgo                                            | Magnitud                                          |
|---|------------------------|-----------------------------------------------------|---------------------------------------------------|
| 0 | Territorial            | Concentración en 5 localidades del sur/suroccidente | 47.4% de los casos en Kennedy, Bosa, Suba, Ciu... |
| 1 | Temporal               | Crecimiento sostenido tras la caída de 2020         | Pico de 7.176 casos en 2025, frente a 3.514 en... |
| 2 | Sexo                   | Los hombres reportan más del doble de casos qu...   | 69.4% hombres frente a 30.6% mujeres              |
| 3 | Entorno de consumo     | La vía pública y la casa de amigos superan por...   | 57.2% vía pública / casa de amigos frente a 11... |
| 4 | Educación              | Mayoría con secundaria incompleta o completa        | 64.6% de los casos                                |
| 5 | Asociación estadística | Todas las variables cruzadas están asociadas a a... | V de Cramér entre 0.03 y 0.12 (p < 0.001 en to... |
| 6 | Modelo predictivo      | Random Forest fue el de mejor desempeño, pero ...   | Accuracy ≈ 58.1%, AUC ≈ 0.621                     |
| 7 | Pronóstico y alerta    | Árbol de Decisión detecta el 100% de los meses...   | Accuracy 66.7%, recall 100% para la clase 'Alto'  |

Esta tabla resume, en un solo lugar, por qué el problema no puede abordarse con una estrategia genérica: el consumo abusivo de SPA en jóvenes bogotanos tiene un patrón territorial, temporal, de género y de entorno bien definido, lo cual es información directamente accionable para el diseño de política pública.

#### 13.2. Sobre el Análisis Estadístico

Un hallazgo metodológico central de este proyecto, más allá de los datos descriptivos, es la diferencia entre significancia estadística y relevancia práctica. Las cuatro pruebas de chi-cuadrado realizadas (nivel educativo-aseguramiento, sexo-localidad de riesgo, nivel educativo-localidad de riesgo, y aseguramiento-localidad de riesgo) mostraron asociaciones estadísticamente significativas en todos los casos, pero con un tamaño de efecto débil en las cuatro (V de Cramér máximo de 0.118).

La razón por la que esto importa es que, con un tamaño de muestra tan grande como el de este estudio (más de 41.000 casos), casi cualquier diferencia entre grupos alcanza significancia estadística, incluso si esa diferencia es pequeña en términos prácticos. Por eso, ninguna de las variables sociodemográficas analizadas por sí sola explica de forma contundente por qué un caso proviene de una localidad de alto riesgo. Esto no invalida los hallazgos, pero sí matiza su alcance: son patrones reales, pero parciales.

#### 13.3. Sobre el Modelo Predictivo

Random Forest fue el modelo con mejor desempeño entre los tres comparados (Accuracy  $\approx$  58.1%, AUC  $\approx$  0.621 en validación cruzada), superando de forma modesta a la regresión logística y al árbol de decisión individual. Este resultado es consistente, y no contradictorio, con lo encontrado en el análisis estadístico: si las asociaciones individuales entre las variables predictoras y la localidad de riesgo son débiles, es esperable que un modelo que las combine todas alcance un desempeño mejor que el azar, pero no un desempeño alto.

La lectura de fondo es que el perfil sociodemográfico individual de un caso (sexo, nivel educativo, aseguramiento, lugar de consumo) no es suficiente para predecir con alta certeza si proviene de una localidad de alto riesgo. Esto sugiere que la localidad de residencia está determinada, en mayor medida, por factores estructurales del territorio (condiciones de vivienda, acceso a servicios públicos y de salud, dinámicas del barrio) que por las características individuales de consumo de la persona. En otras palabras, el modelo no falló por un problema técnico, sino porque el fenómeno que intenta predecir (dónde vive alguien) depende de variables que este dataset no incluye.

El año de notificación y el tipo de aseguramiento fueron las variables más influyentes según Random Forest, aunque con la salvedad metodológica señalada en la sección 11.4: la importancia del año puede estar parcialmente sobrestimada por un sesgo conocido del algoritmo hacia variables numéricas de alta cardinalidad.

### 13.4. Sobre el Pronóstico y la Alerta Temprana

El modelo complementario de la sección 12 aporta una lectura que refuerza, desde otro ángulo, la conclusión anterior: así como el perfil sociodemográfico individual no basta para predecir con precisión la localidad de riesgo, tampoco un solo modelo basta para pronosticar con exactitud el volumen mensual de casos. Sin embargo, a diferencia del modelo de localidad, aquí sí se logró un resultado directamente accionable: Árbol de Decisión y Random Forest identifican el 100% de los meses históricos de alto volumen, lo que demuestra que, aunque el número exacto de casos es difícil de anticipar, el momento en que ocurre un pico sí puede detectarse con buena confiabilidad a partir de patrones temporales simples (mes, año, tendencia). Esto convierte a este modelo complementario en una herramienta más inmediatamente útil para la gestión operativa que el modelo de localidad de riesgo.

## 14. Recomendaciones

A partir de estos hallazgos, se plantean las siguientes recomendaciones, dirigidas a quienes diseñan estrategias de prevención del consumo de SPA en jóvenes en Bogotá:

1. Priorizar la intervención territorial en las cinco localidades identificadas (Kennedy, Bosa, Suba, Ciudad Bolívar y Engativá), dado que concentran casi la mitad de los casos con una fracción menor del territorio. Esto permite optimizar recursos limitados de prevención, dirigiéndolos donde el retorno esperado es mayor.
2. Trasladar el foco de la prevención del entorno escolar hacia los espacios públicos y de socialización no estructurada. El hallazgo de que la vía pública y la casa de amigos superan por mucho al colegio como entornos de consumo implica que los programas centrados únicamente en el aula dejan sin cobertura a los espacios donde el fenómeno realmente ocurre.
3. Diseñar estrategias diferenciadas por sexo, dada la brecha marcada encontrada (69.4% frente a 30.6%), sin dejar de mantener rutas de atención accesibles para mujeres, que pueden enfrentar barreras distintas de acceso a los servicios de salud.
4. Reforzar los programas de retención y nivelación escolar, dado que el 64.6% de los casos corresponde a jóvenes con secundaria incompleta o completa, lo que sugiere una asociación entre rezago o abandono educativo y consumo abusivo que valdría la pena abordar desde el sector educativo, no solo desde el sector salud.
5. Complementar este análisis con variables estructurales del territorio no disponibles en este dataset (por ejemplo, indicadores socioeconómicos por UPZ, cobertura de servicios públicos, o presencia institucional en el barrio), ya que el modelo predictivo demostró que el perfil individual del joven no es suficiente para explicar el riesgo territorial, y es probable que esas variables adicionales aporten el poder predictivo que faltó en este ejercicio.
6. Implementar un sistema de alerta temprana mensual basado en el modelo de Árbol de Decisión de la sección 12, que identificó correctamente el 100% de los meses históricos de alto volumen de casos. Esto permitiría anticipar picos de demanda en los servicios de atención, en lugar de reaccionar únicamente después de que el aumento ya fue reportado.

## 15. Referencias

- Consumo abusivo o problemático de sustancias psicoactivas. Datos Abiertos Bogotá. <https://datosabiertos.bogota.gov.co/dataset/consumo-abusivo-o-problematico-de-sustancias-psicoactivas>
- United Nations. Objetivos de desarrollo sostenible | Naciones Unidas. <https://www.un.org/es/impacto-acad%C3%A9mico/page/objetivos-de-desarrollo-sostenible>
- Universidad EAN. (2026). Material de clase del curso DataXperience: Estadística Descriptiva y Distribuciones, Taller de Buenas Gráficas, Panorama de Modelos de Machine Learning, Panorama Aplicado de Modelos de ML [Material no publicado].
- Observatorio de Drogas de Colombia. ODC - Observatorio de Drogas de Colombia. Ministerio de Justicia y del Derecho. <https://odc.minjusticia.gov.co/odc>
- DANE. Encuesta nacional de consumo de sustancias psicoactivas (ENCSPA). <https://www.dane.gov.co/index.php/estadisticas-por-tema/salud/encuesta-nacional-de-consumo-de-sustancias-psicoactivas-encspa>